UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Facultad de Ingeniería

Carrera de Ingeniería de Software

# BarberTurno

*Sistema web de reservas y turnos para barberías*

## Avance de Proyecto Final 3 (AC-S12-APF3)

Construcción del producto: arquitectura, recursos Java, control de versiones e interfaces

Estudiante

**Sohnny Walter Lima Infanzón**

Curso Integrador I: Sistemas Software

Sección 20058 · Virtual en vivo

Docente

Mg. Ing. Juan Barahona Sánchez

Fecha límite de entrega: 29 de octubre de 2026, 22:30 (hora de Lima)

Versión del informe: 21/10/2026

Ayacucho · Perú · 2026

# Índice

**Resumen** ..... **3**
Cómo leer este informe ..... 3
**1. Continuidad del proyecto: APF1 → APF2 → APF3** ..... **4**
1.1 Correspondencia de identificadores ..... 4
**2. Problema, objetivos y alcance comprometido** ..... **4**
**3. Arquitectura implementada** ..... **6**
3.1 Recorrido de una petición ..... 6
**4. Diseño de la solución: MVC, DAO, SOLID y TDD** ..... **8**
4.1 MVC en una arquitectura SPA + API ..... 8
4.2 DAO con Spring Data JPA ..... 8
4.3 Principios SOLID con clases concretas ..... 10
4.4 TDD: qué se hizo y qué no ..... 11
**5. Seguridad aplicada** ..... **13**
**6. Persistencia, integridad y prevención de solapamientos** ..... **15**
6.1 Por qué no basta con consultar y luego guardar ..... 15
**7. Recursos Java: Logback, Apache POI, Apache Commons y Google Guava** ..... **18**
7.1 Logback ..... 18
7.2 Apache POI ..... 19
7.3 Apache Commons (Commons Lang) ..... 20
7.4 Google Guava ..... 21
7.5 Resumen de dependencias y seguridad en el tratamiento de la información ..... 22
**8. Interfaces gráficas implementadas** ..... **23**
8.1 Cliente ..... 23
8.2 Barbero ..... 26
8.3 Administrador ..... 28
8.4 Estados de error, carga y acceso restringido ..... 34
**9. Matriz requisito → pantalla → implementación → prueba** ..... **35**
**10. Control de versiones con Git y GitHub** ..... **36**
10.1 Evolución por fases ..... 38
10.2 Qué está publicado y qué no ..... 39
**11. Pruebas y resultados técnicos** ..... **40**
11.1 Participación de Java frente al compromiso del APF2 ..... 40
**12. Despliegue en Azure y monitorización** ..... **41**
**13. Estado de aceptación, limitaciones y pendientes** ..... **43**
13.1 Limitaciones conocidas ..... 43
**14. Autoría, uso de asistentes de IA y dominio del código** ..... **44**
**15. Propuesta de sustentación** ..... **45**
**16. Conclusiones** ..... **45**
**Referencias** ..... **46**
**Anexo A. Resumen de la matriz de la rúbrica** ..... **47**
**Anexo B. Inventario de evidencias y reproducción** ..... **47**

# Resumen

El tercer avance de **BarberTurno** demuestra la evolución desde el diseño y el prototipo del APF2 hasta un sistema implementado, probado y desplegado. Se conserva la alternativa seleccionada (**A1**): una SPA en **Angular 22**, una API **Spring Boot 4.1.1 con Java 21** que concentra toda la lógica de negocio y una base **PostgreSQL 18**. El alcance comprometido se mantiene: una barbería de una sede en Huamanga, hasta diez profesionales activos y tres roles (cliente, barbero y administrador).

Los catorce requisitos funcionales del APF2 (RF-01 a RF-14) están implementados con interfaces que funcionan para los tres roles, junto con siete mejoras registradas (RF-15 a RF-21). El código está en un repositorio público de GitHub con 450 commits en `main`, integración continua con GitHub Actions y un historial trazable por tarea (T-01 a T-57). La versión del código `d2f3107` superó **1548** pruebas de backend, **442** de frontend y **48** casos E2E en el último checkpoint (17–18/10/2026), y está desplegada en **Azure for Students** desde el 19/10/2026.

Para este avance se incorporó la tarea **T-58** (exportación del reporte a Excel), que acredita el uso real de **Apache POI**, **Apache Commons Lang** y **Google Guava**, además de **Logback**, que el sistema ya utilizaba. T-58 está implementada con TDD documentado, probada (1578 pruebas de backend, 450 de frontend y 48 E2E) y revisada por el coordinador y por Gemini (veredicto APROBADO). Está integrada en `main` (merge `3bcd5d0`) y publicada en GitHub (CI 39716850704 success); la versión desplegada en Azure sigue siendo `d2f3107`.

El informe distingue la **cobertura funcional implementada** de la **aceptación completa**: siguen pendientes la prueba de usabilidad con cinco participantes (CP-11), la medición mensual de disponibilidad (RNF-02), la prueba funcional mínima en producción y el ensayo de restauración en Azure. El negocio es un caso académico: no se presentan entrevistas, clientes reales ni beneficios económicos.

## Cómo leer este informe

| Criterio de la rúbrica SI53-APF3-2026A | Puntaje | Secciones |
|---|---|---|
| 1. Diseño de la solución (MVC, TDD, DAO, SOLID y seguridad) | 3 | 3, 4, 5 y 6 |
| 2. Uso de recursos Java (Guava, POI, Commons, Logback) | 2 | 7 |
| 3. Control de versiones (Git y GitHub) | 3 | 10 |
| 4. Implementación de las interfaces gráficas | 6 | 8 y 9 |
| 5. Construcción del producto final | 4 | 9, 11, 12, 13 y 14 |
| 6. Sustentación oral | 2 | 15 y material de sustentación |

La matriz completa de la rúbrica, con la evidencia, los archivos y las brechas de cada criterio, se entrega en `Matriz_Rubrica.md` y se resume en el Anexo A.

# 1. Continuidad del proyecto: APF1 → APF2 → APF3

Cada avance construye sobre el anterior sin cambiar el problema, el alcance ni la alternativa elegida. El APF3 no repite el análisis del negocio: muestra cómo el diseño se convirtió en código y con qué evidencias se puede comprobar.

| Avance | Foco | Resultado principal |
|---|---|---|
| APF1 | Contexto, problema, requisitos iniciales y alternativas | Caso académico de la barbería de Huamanga; selección preliminar de una web con backend Java |
| APF2 (01/10/2026) | Canvas, tres alternativas con ocho pantallas cada una, procesos, clases, entidad relación, prototipo HTML y tres clases Java con Javadoc | Alternativa A1 elegida con 4,80 puntos; matriz RF-01…RF-14 → P01…P08; plan de pruebas CP-01…CP-12 |
| APF3 (este avance) | Construcción: arquitectura implementada, patrones, bibliotecas Java, Git/GitHub, interfaces funcionales, pruebas y despliegue | Sistema funcionando y desplegado; evidencias reproducibles por commit |

## 1.1 Correspondencia de identificadores

Los identificadores del APF2 se conservan. Cuando evolucionaron, la tabla indica su equivalente actual.

| APF2 | Implementación actual | Observación |
|---|---|---|
| P01 Acceso y perfil | `/ingresar`, `/registro`, `/privacidad`, `/perfil` (Mi cuenta), `/cambiar-password` | El registro exige teléfono y consentimiento de privacidad (RNF-12) |
| P02 Reserva guiada | `/reservar` | Asistente en tres pasos; el ADMIN reserva en nombre de un cliente (RF-18) |
| P03 Mis citas e historial | `/mis-citas` y diálogo de avisos | Reprogramar y cancelar según los permisos calculados por el servidor |
| P04 Agenda y atención | `/agenda` (BARBERO y ADMIN) | Vista diaria y semanal, transiciones de estado y auditoría (RF-17) |
| P05 Servicios | `/admin/servicios` | Alta, edición y desactivación |
| P06 Barberos | `/admin/barberos` y `/admin/usuarios` | Cuenta de personal con contraseña temporal; restablecer acceso (RF-19) |
| P07 Horarios y bloqueos | `/admin/horarios` | Jornada por intervalos; bloqueo para todo el equipo (RF-20) |
| P08 Reportes | `/admin/reportes` | Conteos por estado, servicio y profesional; historial filtrado; exportación a Excel (T-58) |
| Clases del Anexo B (`ReservaService`, `IntervaloReserva`, `PoliticaReserva`) | `ReglasTemporales`, `PoliticaTransiciones` y `ReservaService` del paquete `reservations` | Las reglas temporales del anexo pasaron al dominio y se prueban con los mismos siete casos |
| CP-01…CP-12 | CP-01…CP-20 | CP-13…CP-19 se añadieron con las mejoras; CP-20 con T-58 |

# 2. Problema, objetivos y alcance comprometido

La barbería coordina las citas por mensajes, llamadas y anotaciones dispersas, sin una disponibilidad única. Eso provoca citas superpuestas, demoras para confirmar y cambios sin trazabilidad.

BarberTurno centraliza servicios, profesionales, horarios y reservas en un sistema web con reglas aplicadas en el servidor.

**Objetivo general.** Implementar un sistema web responsivo que centralice las reservas y turnos de una barbería de Huamanga, permita consultar disponibilidad, gestionar cambios y controlar la agenda diaria con trazabilidad.

| Incluido (implementado) | Fuera del alcance |
|---|---|
| Registro, acceso por roles, perfil y contraseña | Aplicación móvil nativa |
| Catálogo, profesionales, jornadas, descansos y bloqueos | Pagos en línea, caja, inventario y facturación |
| Disponibilidad, reserva, reprogramación y cancelación | Varias sedes |
| Agenda, estados, auditoría, avisos internos, historial y reportes | WhatsApp, SMS y marketing automatizado |

**Propuestas futuras, no implementadas.** La multitenencia (varias barberías), nuevos roles de propietario o de plataforma, la reserva por WhatsApp y el uso de IA para atender clientes son ideas de evolución. No forman parte del APF3 ni se presentan como funciones existentes.

# 3. Arquitectura implementada

BarberTurno es un **monolito modular**: un único proceso Java empaqueta la API y los archivos de la SPA. Con un solo desarrollador, una sede y decenas de reservas al día, un único proceso y una sola base de datos permiten transacciones ACID que cubren la reserva, la auditoría y el aviso en una misma operación, que es lo que exige la integridad de las reservas (RNF-05).

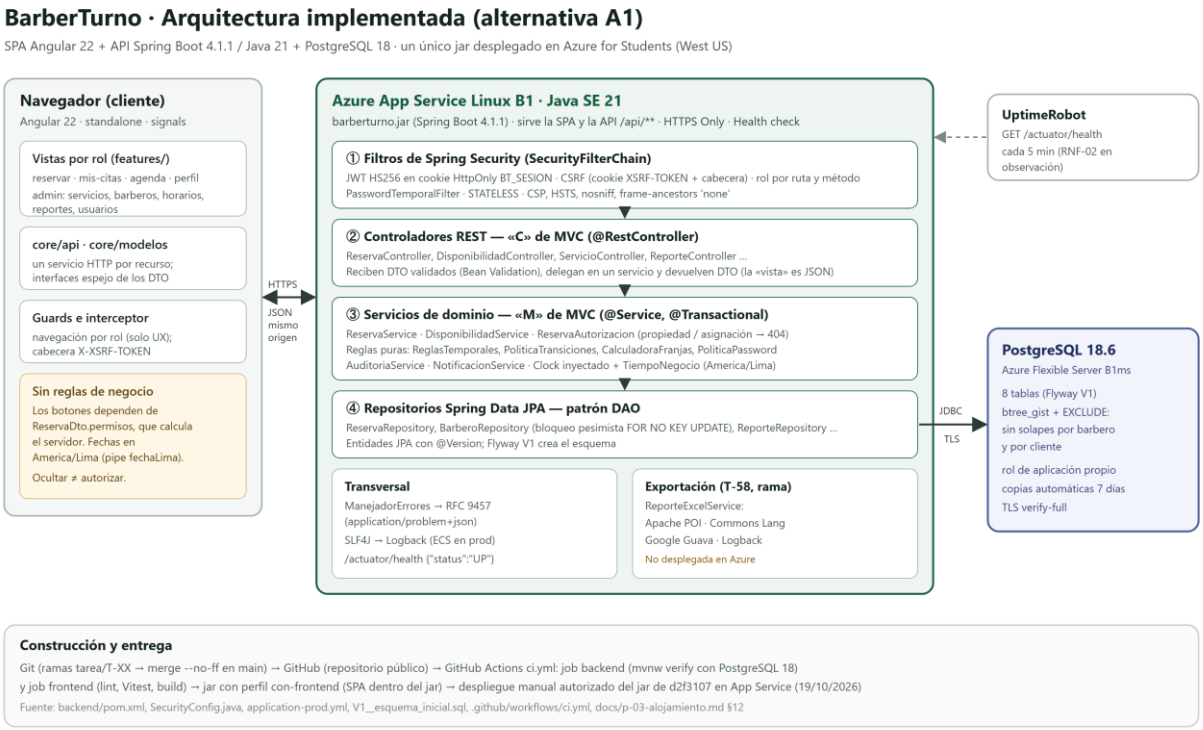



*Figura 1. Arquitectura implementada y despliegue. Elaboración propia a partir del código.*

| Capa | Tecnología | Responsabilidad | Dónde está |
|---|---|---|---|
| Presentación | Angular 22, Angular Material, Tailwind 4 | Pantallas por rol, formularios, navegación; sin reglas de negocio | `frontend/src/app/features` |
| Seguridad | Spring Security 7.1 (resource server JWT) | Autenticación, CSRF, rol por ruta, cabeceras | `common/security` |
| Controladores | Spring Web MVC (`@RestController`) | Recibir la petición, validar el DTO, delegar y responder JSON | `*/…Controller.java` |
| Servicios de dominio | Spring (`@Service`, `@Transactional`) | Reglas de negocio, autorización por propiedad, transacciones | `*/…Service.java`, `reservations/*` |
| Acceso a datos | Spring Data JPA, Hibernate 7.4 | Consultas, bloqueos y persistencia | `*/…Repository.java` |
| Base de datos | PostgreSQL 18 + Flyway | Esquema, restricciones de exclusión, índices | `db/migration/V1__esquema_inicial.sql` |

## 3.1 Recorrido de una petición

Cuando un cliente confirma una reserva ocurre lo siguiente (el diagrama de secuencia de la sección 6 lo muestra en detalle):

1. Angular envía `POST /api/reservas` con el cuerpo JSON, la cookie de sesión `BT_SESION` y la cabecera `X-XSRF-TOKEN`.

2. La cadena de filtros de Spring Security valida el JWT de la cookie, compara el token CSRF de la cabecera con el de la cookie y comprueba que el rol sea CLIENTE o ADMIN. Si falla, responde 401 o 403 sin llegar al controlador.
3. `ReservaController` recibe un `CrearReservaDto` validado con Bean Validation y llama a `ReservaService.crear`.
4. `ReservaService` abre una transacción, bloquea las filas del cliente y del barbero, vuelve a calcular la disponibilidad y guarda la reserva, la auditoría y los avisos. Ante cualquier conflicto revierte todo.
5. El controlador devuelve `201` con un `ReservaDto` que incluye los **permisos** de la reserva; la interfaz habilita los botones según esos permisos, sin reimplementar la política.

# 4. Diseño de la solución: MVC, DAO, SOLID y TDD

La rúbrica pide acreditar al menos tres elementos entre MVC, TDD, DAO y SOLID. Conviene distinguir su naturaleza: **MVC** es un patrón arquitectónico de presentación, **DAO** es un patrón de acceso a datos, **SOLID** es un conjunto de principios de diseño de clases y **TDD** es una práctica de desarrollo. Esta sección muestra dónde se aplica cada uno con código del repositorio.

| Elemento | Tipo | Aplicación en BarberTurno | Grado de evidencia |
|---|---|---|---|
| MVC | Patrón arquitectónico | Controladores REST + servicios/entidades (modelo) + vista Angular que consume JSON | Completo (todo el backend) |
| DAO | Patrón de acceso a datos | 9 interfaces de repositorio Spring Data JPA (una por agregado); ninguna consulta SQL en servicios ni controladores | Completo |
| SOLID | Principios de diseño | SRP, DIP e ISP con clases concretas | Ejemplos concretos |
| TDD | Práctica de desarrollo | Ciclos rojo → verde documentados en T-49 y en T-58; el resto del proyecto usó pruebas automatizadas, no TDD estricto | Parcial y declarado |

## 4.1 MVC en una arquitectura SPA + API

BarberTurno no usa vistas generadas en el servidor (no hay Thymeleaf). MVC se aplica en el backend con **Spring Web MVC**: el `DispatcherServlet` dirige cada petición a un **controlador**; el **modelo** son los servicios de dominio, las entidades y los DTO; y la **vista** es la representación JSON que produce el controlador y que Angular presenta en pantalla. Los controladores no contienen reglas: validan la entrada, obtienen el usuario autenticado y delegan.

**backend/src/main/java/pe/barberturno/reporting/ReporteController.java (d2f3107, Javadoc omitido)**

```
@RestController
public class ReporteController {
    private final ReporteService reportes;

    public ReporteController(ReporteService reportes) { this.reportes = reportes; }

    @GetMapping("/api/reportes/resumen")
    public ResumenReporteDto resumen(@AuthenticationPrincipal UsuarioAutenticado actor,
            @RequestParam(required = false) @DateTimeFormat(iso = ISO.DATE) LocalDate desde,
            @RequestParam(required = false) @DateTimeFormat(iso = ISO.DATE) LocalDate hasta,
            @RequestParam(required = false) Long servicioId, @RequestParam(required = false) Long
                    barberoId,
            @RequestParam Map<String, String> parametros) {
        // ... rechaza parámetros vacíos con 400 VALIDACION
        return reportes.resumen(actor, desde, hasta, servicioId, barberoId);   // delega en el
                modelo
    }
}
```

En el frontend, Angular mantiene la misma separación: los componentes de `features/` son la vista; los servicios de `core/api` encapsulan las llamadas HTTP; y las interfaces de `core/modelos` replican los DTO. Ocultar un botón no autoriza nada: la decisión siempre la toma el servidor.

## 4.2 DAO con Spring Data JPA

Cada agregado tiene una interfaz de repositorio que oculta cómo se accede a PostgreSQL. Los servicios solo conocen métodos con significado de negocio (`buscarSolapamientos`, `bloquearPorIds`, `contarFuturasQueOcupan`); Spring Data genera la implementación en tiempo de ejecución. Así, las

reglas se pueden probar sin escribir SQL en los servicios, y las consultas críticas quedan en un solo lugar.

**reservations/ReservaRepository.java y scheduling/BarberoRepository.java (extracto)**

```
public interface ReservaRepository extends JpaRepository<Reserva, Long>,
        JpaSpecificationExecutor<Reserva> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)                // ③ bloqueo de la fila de la reserva
    @Query("select r from Reserva r where r.id = :id")
    Optional<Reserva> bloquearPorId(@Param("id") Long id);

    @Query("""
            select r from Reserva r
            where r.barbero.id = :barberoId
              and r.estado <> pe.barberturno.reservations.EstadoReserva.CANCELADA
              and r.inicio < :fin and r.fin > :inicio and r.id <> :excluirId
            order by r.inicio, r.id
            """)                                         // intervalos semiabiertos [inicio, fin)
    List<Reserva> buscarSolapamientosExcluyendo(@Param("barberoId") Long barberoId,
            @Param("inicio") Instant inicio, @Param("fin") Instant fin, @Param("excluirId") Long
                    excluirId);
}

public interface BarberoRepository extends JpaRepository<Barbero, Long> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)                // ② serializa las escrituras por barbero
    @Query("select b from Barbero b where b.id in :ids order by b.id")
    List<Barbero> bloquearPorIds(@Param("ids") Collection<Long> ids);
}
```

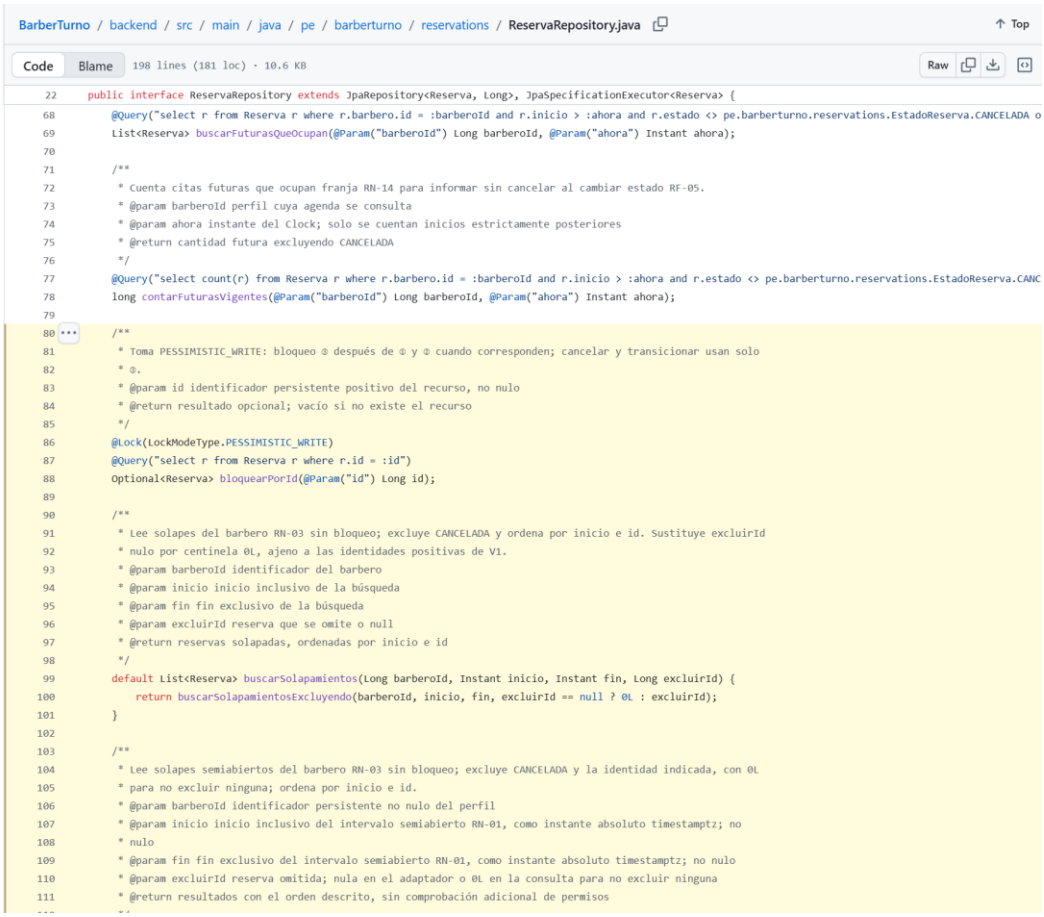



*Figura 2. ReservaRepository en GitHub, commit d2f3107. Captura propia del repositorio público (21/10/2026).*

## 4.3 Principios SOLID con clases concretas

### Responsabilidad única (SRP)

Las reglas que cambian por motivos distintos están en clases distintas. `PoliticaTransiciones` solo decide si un cambio de estado es válido para un rol y un momento; no accede a la base de datos ni a la sesión. `ReservaAutorizacion` solo decide si un actor puede ver una reserva. `AuditoriaService` solo registra cambios. `ReservaService` coordina a todas ellas dentro de una transacción.

**reservations/PoliticaTransiciones.java (validaciones de nulos omitidas)**

```
public final class PoliticaTransiciones {
    public Resultado evaluar(Rol rol, boolean esBarberoAsignado, EstadoReserva actual, EstadoReserva
            destino,
                             Instant ahora, Instant inicio, Duration tolerancia, Duration
                                     anticipacionCliente, String motivo) {
        if (!actual.puedePasarA(destino)) return Resultado.TRANSICION_INVALIDA;          // RN-10
        if (destino == EstadoReserva.CANCELADA) {
            if (rol == Rol.BARBERO) return Resultado.PROHIBIDO;
            return new ReglasTemporales().puedeModificar(ahora, inicio, rol == Rol.ADMIN, motivo,
                    anticipacionCliente)
                    ? Resultado.PERMITIDA : Resultado.FUERA_DE_VENTANA;                 // RN-07 y
                            RN-08
        }
        if (rol != Rol.ADMIN && !(rol == Rol.BARBERO && esBarberoAsignado)) return
                Resultado.PROHIBIDO;  // RN-11
        if (destino == EstadoReserva.EN_ATENCION && ahora.isBefore(inicio.minus(tolerancia))) return
                Resultado.FUERA_DE_VENTANA;
        if (destino == EstadoReserva.NO_ASISTIO && ahora.isBefore(inicio)) return
                Resultado.FUERA_DE_VENTANA;  // RN-12
        return Resultado.PERMITIDA;
    }
}
```

**Problema que resuelve:** la misma política se usa para ejecutar la transición y para calcular `ReservaDto.permisos`, de modo que la interfaz y el servidor no pueden discrepar. Al ser una función pura, se prueba con todas las combinaciones sin levantar Spring.

### Inversión de dependencias (DIP)

Las clases de alto nivel dependen de abstracciones que se inyectan por constructor, nunca de implementaciones creadas con `new`. `ReservaService` recibe repositorios (interfaces), un `Clock` y un `PasswordEncoder`/`UsuarioActual` según el caso. El caso más útil es el reloj: el código de negocio nunca llama a `Instant.now()`; usa el `Clock` inyectado, que en producción es el reloj real de Lima y en las pruebas y la demostración es un reloj fijo (28/09/2026 09:00).

**common/time/ClockConfig.java y constructor de reservations/ReservaService.java**

```
@Bean
public Clock clock(Environment entorno) {
    String fijo = entorno.getProperty("barberturno.reloj-fijo");
    if (fijo != null && entorno.acceptsProfiles(Profiles.of("prod"))) {
        throw new IllegalStateException("barberturno.reloj-fijo no está permitido en prod.");
    }
    if (fijo != null && !fijo.isBlank() && entorno.acceptsProfiles(Profiles.of("demo", "test"))) {
        return Clock.fixed(OffsetDateTime.parse(fijo).toInstant(), TiempoNegocio.ZONA);
    }
    return Clock.system(TiempoNegocio.ZONA);
}


public ReservaService(ReservaRepository reservas, UsuarioRepository usuarios, BarberoRepository
        barberos,
        ServicioRepository servicios, DisponibilidadService disponibilidad, AuditoriaService
                auditoria,
        NotificacionService notificaciones, Clock clock, ParametrosReserva parametros,
                ReservaAutorizacion autorizacion)
```

### Segregación de interfaces (ISP)

Los servicios que solo necesitan saber quién es el usuario actual dependen de `UsuarioActual`, una interfaz de tres métodos, y no de todo el `SecurityContextHolder` de Spring. La implementación `UsuarioActualSecurity` traduce el contexto de seguridad; en una prueba basta con una implementación mínima.

**common/security/UsuarioActual.java**

```
public interface UsuarioActual {
    long id();
    Rol rol();
    Optional<Long> barberoId();
}
```

## 4.4 TDD: qué se hizo y qué no

Tener muchas pruebas no demuestra TDD. TDD exige escribir primero una prueba que falla, después el código mínimo que la hace pasar y, finalmente, refactorizar. En BarberTurno la mayoría de las tareas se entregaron con la implementación y sus pruebas en el mismo ciclo (pruebas automatizadas, no TDD estricto). Hay dos evidencias verificables del ciclo rojo → verde:

| Tarea | Prueba en rojo | Corrección en verde | Evidencia |
|---|---|---|---|
| T-49 (corrección del 403 de Firefox, CSRF) | e9ddeed y 21f86ad: pruebas que reproducen el defecto; la segunda «fallaba 3/4 antes» contra Tomcat real | c669276 y b60cbdc | docs/pruebas/t-49.md y docs/tareas.md |
| T-58 (exportación a Excel) | c95fd3e generador: 12 de 12 fallan por aserción · 564e7b7 endpoint: 14 de 17 · f3bb9b3 frontend: 8 nuevas fallan | 27e6258 · dc8c82c · 9cfd521 | docs/pruebas/t-58.md (rama de T-58) |

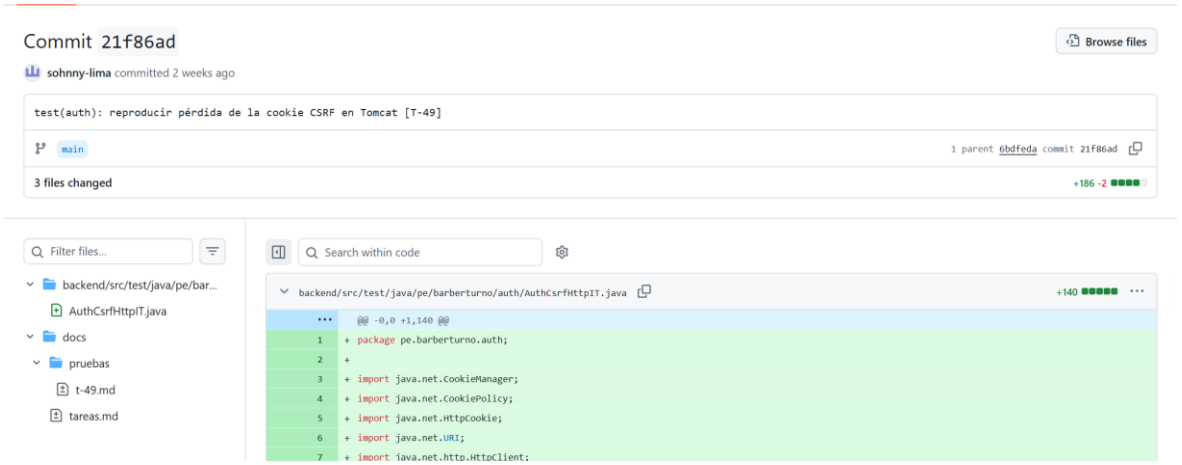



*Figura 3. T-49 en GitHub, paso rojo: commit 21f86ad añade la prueba AuthCsrfHttpIT que reproduce el defecto. Captura propia.*

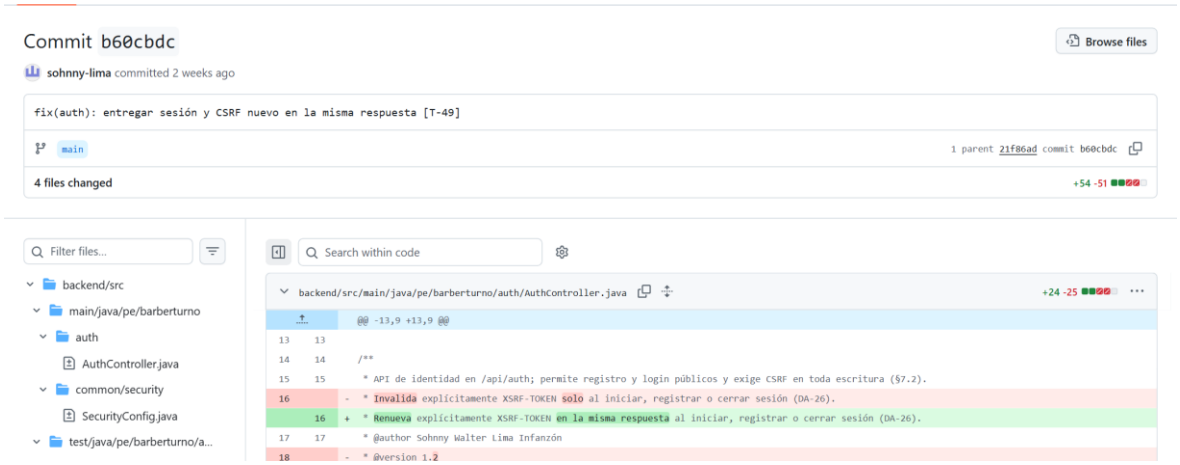



*Figura 4. T-49 en GitHub, paso verde: commit b60cbdc corrige el defecto y la prueba pasa. Captura propia.*

# 5. Seguridad aplicada

La seguridad se diseñó en el servidor: la interfaz puede ocultar opciones, pero cada operación se autoriza en Java. La tabla resume cada control con su implementación y la prueba que lo comprueba.

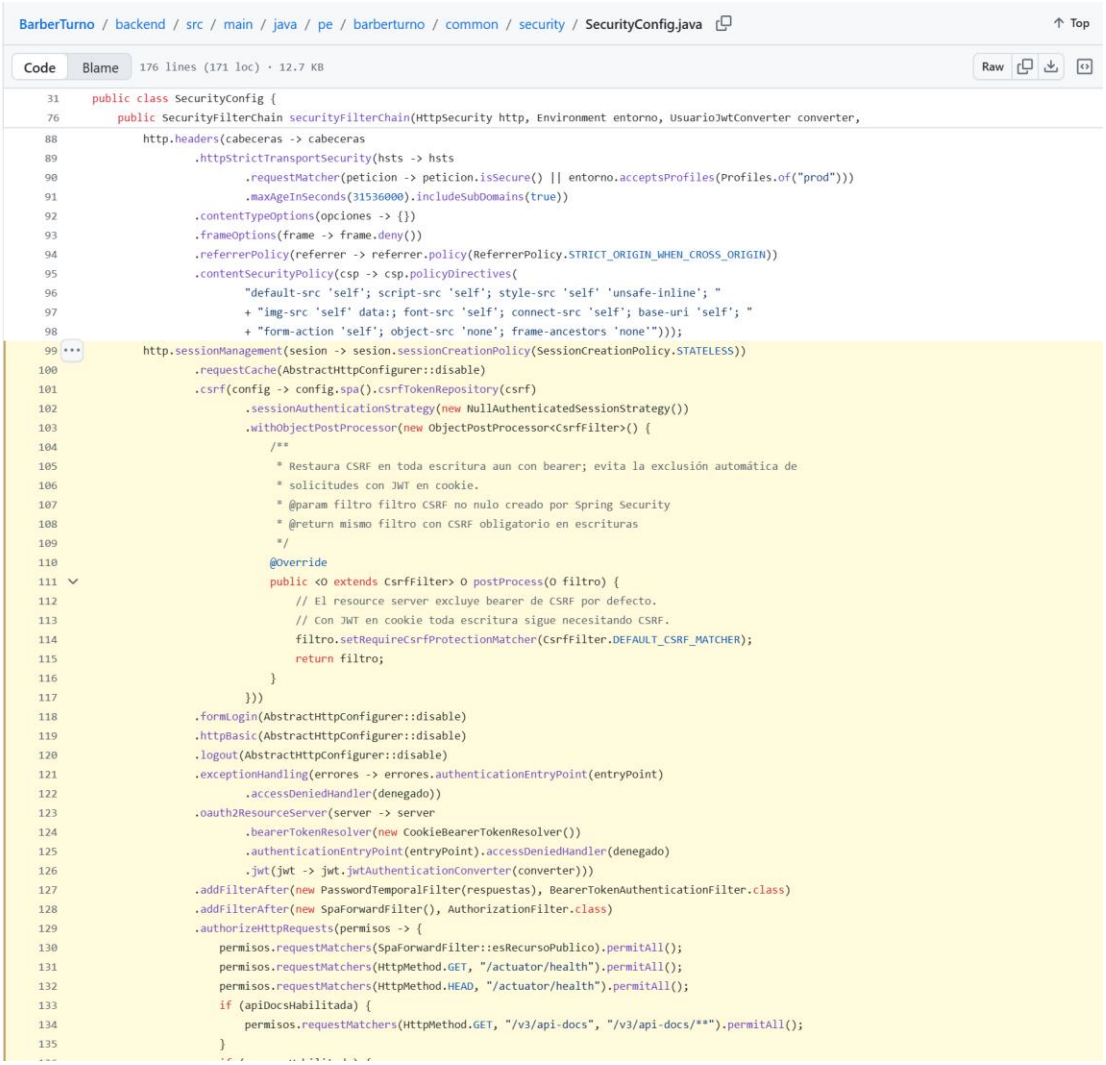
| Control | Implementación | Evidencia |
|---|---|---|
| Contraseñas | BCrypt con coste 12 (`JwtConfig.passwordEncoder`); 8–72 caracteres y máximo 72 bytes UTF-8 (RN-25) | `PoliticaPasswordTest`, `AuthIT` |
| Sesión | JWT HS256 de 8 h en la cookie `BT_SESION` HttpOnly, `SameSite=Strict` y `Secure` en producción; API `STATELESS`; `token_version` revoca sesiones al cambiar la contraseña | `AuthIT`, CP-17 |
| CSRF | Cookie `XSRF-TOKEN` + cabecera `X-XSRF-TOKEN` en toda escritura, también con JWT (`DEFAULT_CSRF_MATCHER`); token estable durante la sesión y renovado al iniciar o cerrar sesión (DA-26, T-49) | Pruebas «sesión válida sin CSRF → 403» y «CSRF incorrecto → 403» en cada escritura |
| Fuerza bruta | Tras 5 intentos fallidos la cuenta se bloquea 15 min; mensaje genérico; comparación con un hash ficticio cuando el correo no existe | CP-14 |
| Autorización por rol | Reglas por ruta y método en `SecurityFilterChain` y `denyAll()` para lo no listado | Matriz de permisos (arquitectura §7.2) |
| Autorización por propiedad | `ReservaAutorizacion`: una reserva ajena responde 404 sin revelar que existe | CP-02, `ReservaConsultaIT` |
| Validación | Bean Validation en los DTO (`record`), restricciones CHECK en la base y errores RFC 9457 sin trazas | `ManejadorErroresTest`, `ErroresMvcIT` |
| Integridad | Transacciones, bloqueos ordenados y `EXCLUDE USING gist` (sección 6) | CP-03, CP-05, CP-13 |
| Cabeceras | HSTS, CSP restrictiva, `X-Frame-Options: DENY`, `nosniff`, `Referrer-Policy` | `CabecerasSeguridadIT`; comprobado en Azure (§12) |
| HTTPS | Lo termina Azure App Service (*HTTPS Only*, HTTP → 301) | Comprobado el 19/10/2026 y el 21/10/2026 |
| Secretos | Variables de entorno; `.local/` ignorado por Git; el perfil `prod` exige las variables y rechaza `demo` y el reloj fijo | `.gitignore`, `application-prod.yml` |
| Registros | Sin contraseñas, JWT, cookies ni datos personales completos; formato ECS en producción | Pruebas con `ListAppender` (§7.1) |

**common/security/SecurityConfig.java (extracto)**

```
http.sessionManagement(sesion -> sesion.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .csrf(config -> config.spa().csrfTokenRepository(csrf)
        .sessionAuthenticationStrategy(new NullAuthenticatedSessionStrategy())
        .withObjectPostProcessor(new ObjectPostProcessor<CsrfFilter>() {
            @Override
            public <O extends CsrfFilter> O postProcess(O filtro) {
                // El resource server excluye bearer de CSRF por defecto.
                // Con JWT en cookie toda escritura sigue necesitando CSRF.
                filtro.setRequireCsrfProtectionMatcher(CsrfFilter.DEFAULT_CSRF_MATCHER);
                return filtro;
            }
        }));
```

**Por qué esta corrección importa:** el soporte de JWT de Spring Security asume que el token viaja en una cabecera `Authorization`, que un sitio atacante no puede enviar, y por eso desactiva CSRF. En BarberTurno el JWT viaja en una cookie, que el navegador sí envía automáticamente; sin esta línea, un formulario de otro sitio podría ejecutar escrituras con la sesión del usuario. La regresión se detectó

en la revisión de T-10/T-11 y desde entonces cada endpoint de escritura tiene pruebas sin CSRF y con CSRF incorrecto.



BarberTurno / backend / src / main / java / pe / barberturno / common / security / SecurityConfig.java ↑ Top

Code Blame 176 lines (171 loc) · 12.7 KB Raw

```
31  public class SecurityConfig {
76      public SecurityFilterChain securityFilterChain(HttpSecurity http, Environment entorno, UsuarioJwtConverter converter,
88          http.headers(cabeceras -> cabeceras
89                  .httpStrictTransportSecurity(hsts -> hsts
90                          .requestMatcher(peticion -> peticion.isSecure() || entorno.acceptsProfiles(Profiles.of("prod")))
91                          .maxAgeInSeconds(31536000).includeSubDomains(true))
92                  .contentTypeOptions(opciones -> {})
93                  .frameOptions(frame -> frame.deny())
94                  .referrerPolicy(referrer -> referrer.policy(ReferrerPolicy.STRICT_ORIGIN_WHEN_CROSS_ORIGIN))
95                  .contentSecurityPolicy(csp -> csp.policyDirectives(
96                          "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; "
97                          + "img-src 'self' data:; font-src 'self'; connect-src 'self'; base-uri 'self'; "
98                          + "form-action 'self'; object-src 'none'; frame-ancestors 'none'")));
99          http.sessionManagement(sesion -> sesion.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
100                 .requestCache(AbstractHttpConfigurer::disable)
101                 .csrf(config -> config.spa().csrfTokenRepository(csrf)
102                         .sessionAuthenticationStrategy(new NullAuthenticatedSessionStrategy())
103                         .withObjectPostProcessor(new ObjectPostProcessor<CsrfFilter>() {
104                             /**
105                              * Restaura CSRF en toda escritura aun con bearer; evita la exclusión automática de
106                              * solicitudes con JWT en cookie.
107                              * @param filtro filtro CSRF no nulo creado por Spring Security
108                              * @return mismo filtro con CSRF obligatorio en escrituras
109                              */
110                             @Override
111                             public <O extends CsrfFilter> O postProcess(O filtro) {
112                                 // El resource server excluye bearer de CSRF por defecto.
113                                 // Con JWT en cookie toda escritura sigue necesitando CSRF.
114                                 filtro.setRequireCsrfProtectionMatcher(CsrfFilter.DEFAULT_CSRF_MATCHER);
115                                 return filtro;
116                             }
117                         }))
118                 .formLogin(AbstractHttpConfigurer::disable)
119                 .httpBasic(AbstractHttpConfigurer::disable)
120                 .logout(AbstractHttpConfigurer::disable)
121                 .exceptionHandling(errores -> errores.authenticationEntryPoint(entryPoint)
122                         .accessDeniedHandler(denegado))
123                 .oauth2ResourceServer(server -> server
124                         .bearerTokenResolver(new CookieBearerTokenResolver())
125                         .authenticationEntryPoint(entryPoint).accessDeniedHandler(denegado)
126                         .jwt(jwt -> jwt.jwtAuthenticationConverter(converter)))
127                 .addFilterAfter(new PasswordTemporalFilter(respuestas), BearerTokenAuthenticationFilter.class)
128                 .addFilterAfter(new SpaForwardFilter(), AuthorizationFilter.class)
129                 .authorizeHttpRequests(permisos -> {
130                     permisos.requestMatchers(SpaForwardFilter::esRecursoPublico).permitAll();
131                     permisos.requestMatchers(HttpMethod.GET, "/actuator/health").permitAll();
132                     permisos.requestMatchers(HttpMethod.HEAD, "/actuator/health").permitAll();
133                     if (apiDocsHabilitada) {
134                         permisos.requestMatchers(HttpMethod.GET, "/v3/api-docs", "/v3/api-docs/**").permitAll();
135                     }
```

*Figura 5. Reglas de autorización por ruta y método en SecurityConfig (GitHub, d2f3107). Captura propia.*

# 6. Persistencia, integridad y prevención de solapamientos

El esquema se crea con Flyway (`V1__esquema_inicial.sql`) y se valida contra las entidades JPA al arrancar. Las fechas se guardan como instantes (`timestamptz`) y se presentan siempre en America/Lima. Los servicios y profesionales no se borran: se desactivan y conservan su historial.

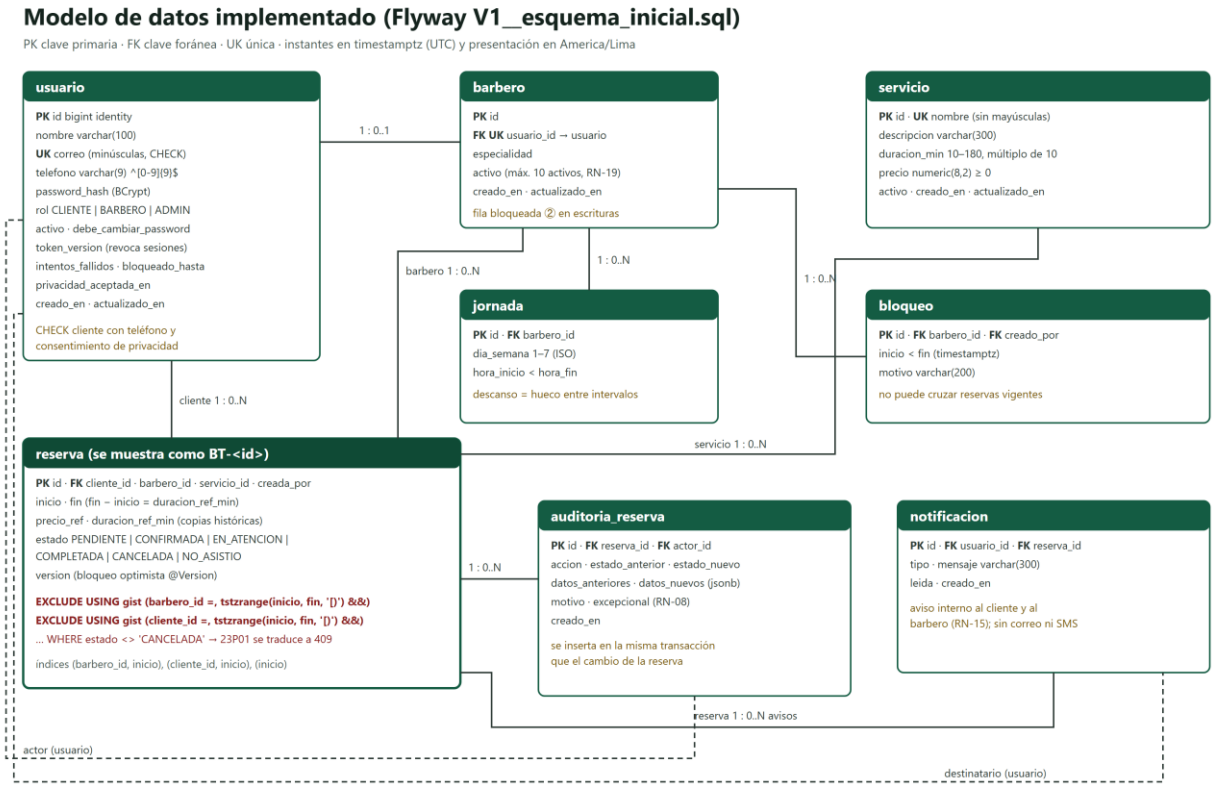



*Figura 6. Modelo de datos implementado (ocho tablas). Elaboración propia a partir de la migración V1.*

## 6.1 Por qué no basta con consultar y luego guardar

Dos clientes pueden consultar la misma franja libre antes de que alguno guarde. Si cada uno comprueba y después inserta, ambos verían la franja libre y se crearían dos citas superpuestas. BarberTurno lo evita con tres capas complementarias:

1. **Serialización por barbero (bloqueo pesimista).** Toda escritura que afecta la disponibilidad de un barbero bloquea primero su fila (`FOR NO KEY UPDATE`) y **después** vuelve a comprobar la disponibilidad dentro de la misma transacción. La segunda solicitud espera y, cuando continúa, ve la reserva confirmada por la primera.
2. **Red de seguridad en la base de datos.** Dos restricciones `EXCLUDE USING gist` impiden que un barbero o un cliente tengan dos reservas activas que se solapen, aunque algún camino de código omitiera el protocolo. El error `23P01` se traduce a un 409 comprensible.
3. **Bloqueo optimista.** El campo `version` (`@Version`) detecta acciones sobre un estado desactualizado, por ejemplo un doble clic o dos pestañas: responde 409 `VERSION_DESACTUALIZADA`.

Para evitar interbloqueos, todas las operaciones toman los bloqueos en el mismo orden: ① fila del cliente → ② filas de barberos por id ascendente → ③ fila de la reserva.

**db/migration/V1__esquema_inicial.sql**

```
CONSTRAINT reserva_sin_solape_barbero EXCLUDE USING gist
  (barbero_id WITH =, tstzrange(inicio, fin, '[)') WITH &&) WHERE (estado <> 'CANCELADA'),
CONSTRAINT reserva_sin_solape_cliente EXCLUDE USING gist
  (cliente_id WITH =, tstzrange(inicio, fin, '[)') WITH &&) WHERE (estado <> 'CANCELADA')
```

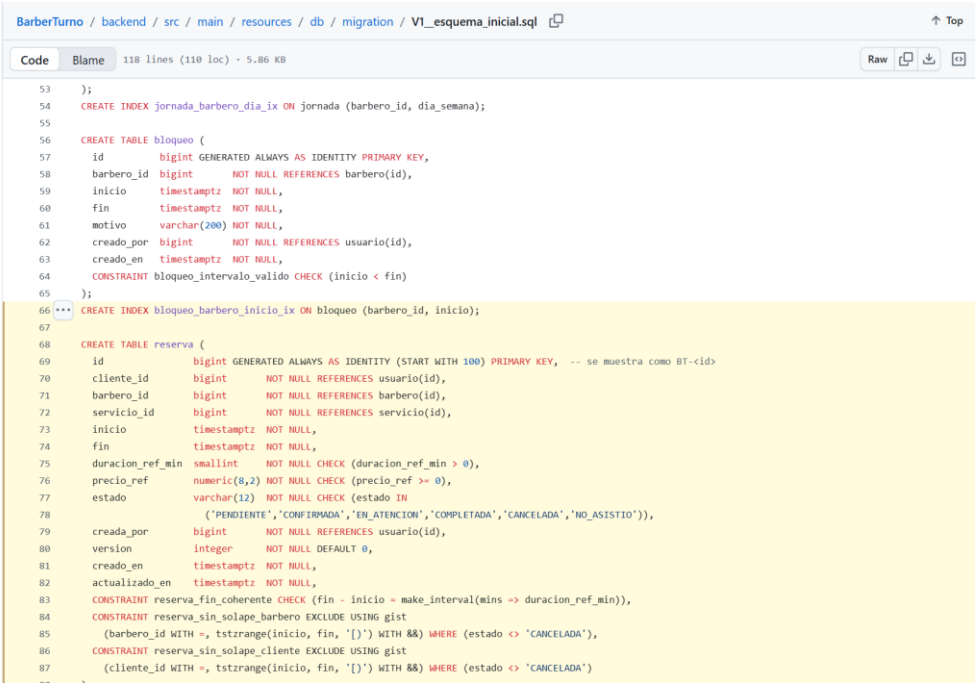



*Figura 7. Restricciones de exclusión en la migración V1 publicada en GitHub (d2f3107). Captura propia.*

**reservations/ReservaService.java, método crear (extracto)**

```
var cliente = usuarios.bloquearPorId(asistida ? cmd.clienteId() :
        actor.id()).orElseThrow(this::noEncontrado); // ①
var perfiles = barberos.bloquearPorIds(List.of(cmd.barberoId()));
        // ②
// ... servicio y barbero activos
Instant fin = new ReglasTemporales().calcularFin(inicio, servicio.getDuracionMin());
disponibilidad.validarFranja(barbero.getId(), inicio, fin, 0L).ifPresent(codigo -> {
    throw new NegocioException(codigo, "La franja solicitada no cumple la disponibilidad.");
          // → ROLLBACK
});
if (!reservas.buscarSolapamientosCliente(cliente.getId(), inicio, fin, 0L).isEmpty()) { /* 409 RN-04
        */ }
var reserva = reservas.saveAndFlush(new Reserva(cliente, barbero, servicio, inicio, estado, creador,
        ahora));
auditoria.registrarCambio(reserva, creador, AccionAuditoria.CREAR, /* ... */);
        // misma transacción
```

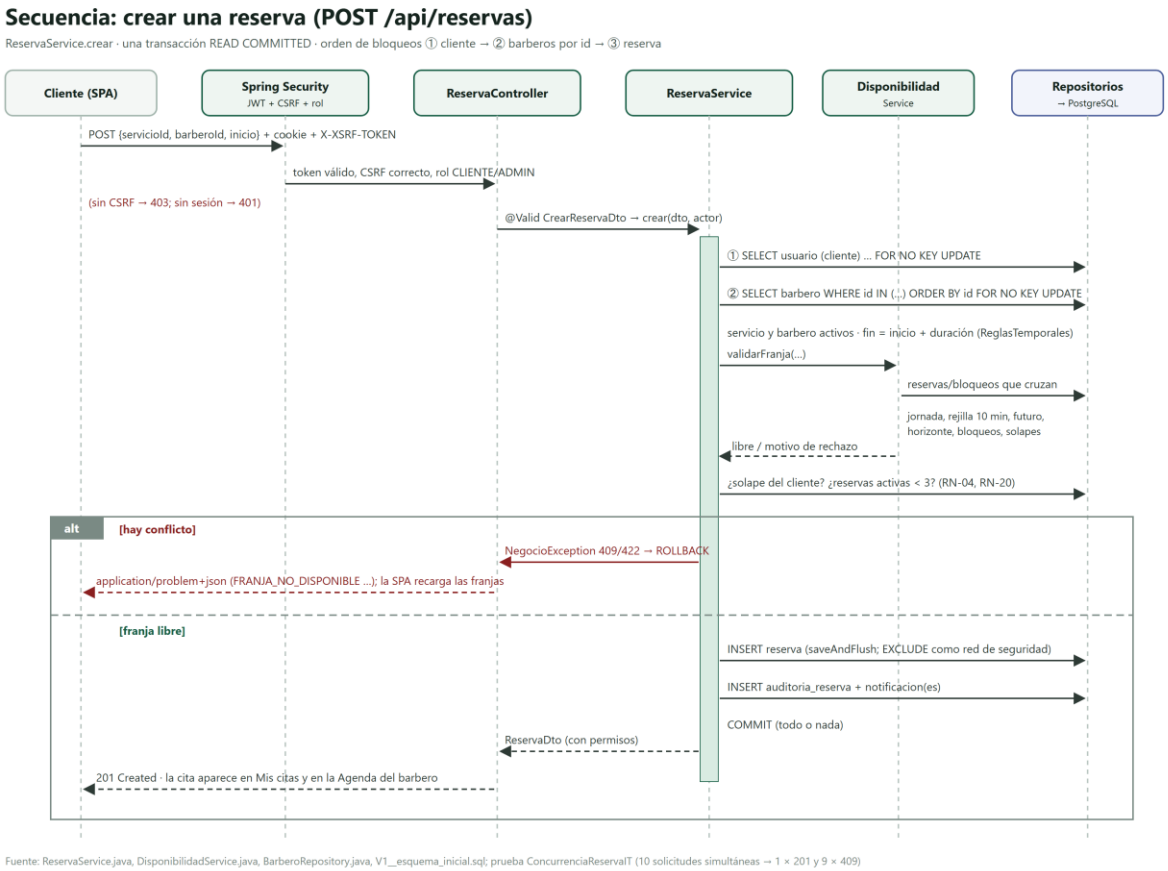



*Figura 8. Secuencia de creación de una reserva con el control de concurrencia. Elaboración propia a partir del código.*

**Evidencia:** `ConcurrenciaReservaIT` envía 10 solicitudes simultáneas a la misma franja contra PostgreSQL real y comprueba que exactamente una recibe 201 y nueve reciben 409; también cruza una reserva con un bloqueo simultáneo y una reprogramación con una creación (CP-03, CP-05). `EsquemaIT` comprueba directamente las restricciones `EXCLUDE` y que dos citas contiguas (10:00–10:30 y 10:30–11:00) sí se aceptan (CP-04).

# 7. Recursos Java: Logback, Apache POI, Apache Commons y Google Guava

La rúbrica pide incorporar Google Guava, Apache POI, Apache Commons y Logback, y el docente aclaró que no basta con declararlas: hay que mostrar dónde se aplican y qué aportan. La auditoría del 21/10/2026 sobre `pom.xml` y el árbol de dependencias efectivo encontró lo siguiente **antes** de este avance:

| Biblioteca | Situación en d2f3107 | Uso en el código |
|---|---|---|
| Logback 1.5.38 | Transitiva (spring-boot-starter-logging) | Sí: backend de los registros SLF4J; formato ECS en producción; pruebas con `ListAppender` |
| Apache Commons | `commons-lang3` 3.20.0 y `commons-logging` 1.3.6, solo transitivas | No |
| Google Guava | Ausente | No |
| Apache POI | Ausente | No |

Una dependencia transitiva no acredita un uso funcional. Por eso se propuso al responsable, y este autorizó como excepción puntual al freeze, la tarea **T-58**: exportar a Excel el reporte que el administrador ya consulta (RF-14). Concentra las tres bibliotecas que faltaban en una función útil y acotada, sin alterar reglas de negocio ni dispersar dependencias por el dominio (MJ-22, DA-29).

> **Versión entregable y versión desplegada.** El código que acompaña este informe es el de `main` en el commit `3bcd5d0` (merge de T-58; CI 39716850704 success), que incluye T-58. Los materiales de esta entrega se publicaron después, en commits solo de documentación registrados en `docs/cierre.md`. La versión desplegada en Azure es `d2f3107`, **sin** la exportación a Excel: el responsable decidió no redesplegar por ahora, así que la demostración del Excel se hace en local.

## 7.1 Logback

Spring Boot usa **SLF4J** como fachada (las clases escriben `LoggerFactory.getLogger(...)`) y **Logback** como implementación que formatea y escribe los eventos. Logback llega con `spring-boot-starter-logging` (1.5.38, versión gestionada por Boot 4.1.1); no hay `logback-spring.xml` porque la configuración se hace con propiedades de Spring Boot.

| Aspecto | Detalle |
|---|---|
| Artefacto | `ch.qos.logback:logback-classic:1.5.38` (transitiva) |
| Configuración | `application.yml` (niveles), `application-dev.yml` (`pe.barberturno: DEBUG`), `application-prod.yml` (`logging.structured.format.console: ecs`) |
| Uso | `AuthService` (intentos fallidos y bloqueos por id), `ManejadorErrores` (rechazos y errores no controlados), `JwtConfig`, `DatosDemoRunner`, `ReporteExcelService` (exportaciones) |
| Seguridad | Solo se registran identificadores numéricos y códigos; nunca contraseñas, JWT, cookies, correos ni nombres. Las pruebas lo verifican capturando los eventos de Logback |

**application-prod.yml y auth/AuthService.java**

```
logging:
  structured:
    format:
      console: ecs            # application-prod.yml: JSON estructurado (Elastic Common Schema)


// AuthService.login
LOG.info("Login fallido de usuario id={}.", usuario.getId());       // solo el id, nunca el correo
if (bloquear) LOG.warn("Bloqueo temporal de usuario id={}.", usuario.getId());
```

**test/.../common/error/ManejadorErroresTest.java: la prueba usa la API de Logback para comprobar que el registro no filtra datos**

```
Logger logger = (Logger) LoggerFactory.getLogger(ManejadorErrores.class);   //
        ch.qos.logback.classic.Logger
ListAppender<ILoggingEvent> captura = new ListAppender<>();
captura.start();
logger.addAppender(captura);
// ... se provoca un error 4xx con datos ficticios en la petición
var evento = captura.list.getFirst();
assertThat(evento.getLevel()).isEqualTo(Level.WARN);
assertThat(evento.getThrowableProxy()).isNull();                          // sin traza
assertThat(evento.getFormattedMessage()).contains("HTTP " + estado)
        .doesNotContain("secreto-ficticio", "persona@ejemplo.test", "Exception");
```

## 7.2 Apache POI

Apache POI es la biblioteca Java de referencia para leer y escribir documentos de Microsoft Office. En BarberTurno genera el archivo `.xlsx` del reporte que el administrador ya ve en pantalla (RF-14). `ReporteExcelService.generar` crea un `XSSFWorkbook` en memoria, escribe la hoja «Resumen» y la serializa a bytes; el controlador la entrega como descarga.

| Aspecto | Detalle |
|---|---|
| Artefacto | `org.apache.poi:poi-ooxml:5.5.1` (directa, versión verificada en Maven Central el 21/10/2026) |
| Dónde | `reporting/ReporteExcelService.java`, métodos `generar` y `fila`; endpoint GET `/api/reportes/resumen.xlsx` en `ReporteController.excel` |
| Aporte | El administrador conserva o comparte el resumen fuera del sistema con las mismas cifras que la pantalla; los números se escriben como celdas numéricas para poder sumarlos en Excel |
| Seguridad | Solo ADMIN; solo datos agregados (nombres de servicios y profesionales, sin clientes ni reservas individuales); nunca se escriben fórmulas (`setCellFormula` no se usa); *try-with-resources* libera el libro; `Cache-Control: no-store` |

**reporting/ReporteExcelService.java, método generar (extracto)**

```
try (var libro = new XSSFWorkbook(); var salida = new ByteArrayOutputStream()) {
    var hoja = libro.createSheet("Resumen");
    var textoSeguro = libro.createCellStyle();
    textoSeguro.setQuotePrefixed(true);                // estilo OOXML «texto», ver 7.3
    fila(hoja, textoSeguro, "Reporte de reservas · BarberTurno", null);
    fila(hoja, textoSeguro, "Fecha de generación (Lima)",
            FECHA.format(TiempoNegocio.aLima(reloj.instant())));
    fila(hoja, textoSeguro, "Total", resumen.total());
    for (var estado : EstadoReserva.values()) {
        fila(hoja, textoSeguro, etiqueta(estado), resumen.porEstado().getOrDefault(estado, 0L));
    }
    for (var grupo : resumen.porServicio()) fila(hoja, textoSeguro, grupo.nombre(), grupo.total());
    for (var grupo : resumen.porBarbero()) fila(hoja, textoSeguro, grupo.nombre(), grupo.total());
    libro.write(salida);
    return salida.toByteArray();
}

private void fila(Sheet hoja, CellStyle textoSeguro, String etiqueta, Object valor) {
    var fila = hoja.createRow(hoja.getPhysicalNumberOfRows());
    texto(fila.createCell(0), etiqueta, textoSeguro);
    if (valor instanceof Long numero) fila.createCell(1).setCellValue(numero.doubleValue());   //
            celda numérica
    else if (valor instanceof String cadena) texto(fila.createCell(1), cadena, textoSeguro);
}
```

La prueba `ReporteExcelServiceTest.contenidoCompletoNumericoSinFormulasNiHojasOcultas` vuelve a abrir el libro con POI y comprueba el contenido, que el total es numérico y que **ninguna** celda es de tipo fórmula. `ReporteExcelIT.adminDescargaMismasCifrasQueJsonConYSinFiltros` descarga el archivo por HTTP contra PostgreSQL y compara cada cifra con `GET /api/reportes/resumen` para los mismos filtros.

## 7.3 Apache Commons (Commons Lang)

Apache Commons es una familia de bibliotecas; se usa **Commons Lang** (`commons-lang3`), que amplía las utilidades de `java.lang`. Antes de T-58 solo llegaba como dependencia transitiva y no se usaba; ahora se declara de forma directa (con la versión que gestiona Spring Boot) y cumple una función de seguridad concreta.

**Problema que resuelve: inyección de fórmulas.** Si un texto empieza por `=`, `+`, `-` o `@` (o por tabulación o retorno de carro), una hoja de cálculo puede interpretarlo como fórmula cuando el usuario edita la celda; OWASP lo describe como *CSV/Formula Injection*. Los nombres de servicios y profesionales los escribe un administrador, pero el archivo puede reenviarse. `StringUtils.startsWithAny` detecta esos prefijos y la celda recibe el estilo `quotePrefixed` de OOXML, que obliga a Excel a tratarla como texto. **El valor no se modifica**: no se añade un apóstrofo ni se borran caracteres.

| Aspecto | Detalle |
|---|---|
| Artefacto | `org.apache.commons:commons-lang3:3.20.0` (directa; versión gestionada por Spring Boot 4.1.1) |
| Dónde | `ReporteExcelService.texto` |
| Aporte | Una sola expresión, legible y probada, para la lista de prefijos peligrosos |
| Seguridad | Defensa en profundidad para la información exportada; la prueba parametrizada cubre los seis prefijos |

**reporting/ReporteExcelService.java, método texto**

```
private void texto(Cell celda, String valor, CellStyle textoSeguro) {
    celda.setCellValue(valor);                          // siempre celda de texto, valor intacto
    if (StringUtils.startsWithAny(valor, "=", "+", "-", "@", "\t", "\r"))
            celda.setCellStyle(textoSeguro);
}
```

**ReporteExcelServiceTest.java: prueba parametrizada**

```
@ParameterizedTest @ValueSource(strings = {"=SUM(A1:A2)", "+Corte", "-Corte", "@Corte", "\tCorte",
        "\rCorte"})
void conservaTextoDeRiesgoYActivaQuotePrefixed(String nombre) throws Exception {
    // ...
    assertThat(celda.getCellType()).isEqualTo(CellType.STRING);
    assertThat(celda.getStringCellValue()).isEqualTo(nombre);          // el dato no cambia
    assertThat(celda.getCellStyle().getQuotePrefixed()).isTrue();      // pero se trata como texto
}
```

## 7.4 Google Guava

Guava es la biblioteca de utilidades de Google para Java. En T-58 cumple dos funciones: **verificar una invariante del negocio antes de entregar el archivo** y **medir la generación** para el registro técnico.

`Preconditions.checkState` comprueba la regla de RF-14: la suma por estado, la suma por servicio y la suma por profesional deben coincidir con el total. Si no coinciden, no se genera un archivo incoherente: se registra una advertencia sin datos y la API responde 500 genérico, sin archivo. `Stopwatch` mide el tiempo con un `Ticker` inyectable, de modo que la prueba usa un reloj monotónico simulado (125 ms exactos) en lugar de depender del tiempo real.

| Aspecto | Detalle |
|---|---|
| Artefacto | `com.google.guava:guava:33.7.2-jre` (directa; publicada el 29/09/2026, verificada en Maven Central el 21/10/2026) |
| Dónde | `ReporteExcelService.generar` (`Preconditions`, `Stopwatch`) y su constructor (`Ticker`) |
| Aporte | Integridad del archivo exportado y medición testeable de una operación más costosa que el JSON |
| Seguridad | Evita entregar información inconsistente; la duración se registra sin datos personales |

**reporting/ReporteExcelService.java (extracto)**

```
Preconditions.checkState(
        resumen.porEstado().values().stream().mapToLong(Long::longValue).sum() == resumen.total()
        && resumen.porServicio().stream().mapToLong(ConteoReporteDto::total).sum() ==
                resumen.total()
        && resumen.porBarbero().stream().mapToLong(ConteoReporteDto::total).sum() ==
                resumen.total(),
        "Las agrupaciones del reporte no concilian.");
var cronometro = Stopwatch.createStarted(ticker);        // Ticker inyectable: real en producción,
        simulado en pruebas
// ... generación del libro
LOG.info("Reporte Excel generado: actor={} filas={} bytes={} duracionMs={}", actorId,
        hoja.getPhysicalNumberOfRows(), archivo.length,
                cronometro.stop().elapsed(TimeUnit.MILLISECONDS));
```

Esta misma línea de registro es la aplicación de **Logback** en T-58: `ReporteExcelServiceTest.infoSoloIncluyeActorFilasBytesYDuracionMonotonica` captura el evento con `ListAppender` y comprueba que el mensaje es exactamente `actor=7 filas=18 bytes=… duracionMs=125`, aunque el resumen contenga «=Nombre privado» y un correo: ningún dato personal llega al registro.

## 7.5 Resumen de dependencias y seguridad en el tratamiento de la información

| Biblioteca | Artefacto y versión | Tipo | Clase y método | Aporte | Seguridad |
|---|---|---|---|---|---|
| Logback | logback-classic 1.5.38 | Transitiva (Boot) | `AuthService.login`, `ManejadorErrores`, `ReporteExcelService.generar` | Registro estructurado (ECS en producción) | Sin contraseñas, tokens, correos ni nombres; probado con `ListAppender` |
| Apache POI | poi-ooxml 5.5.1 | Directa (T-58) | `ReporteExcelService.generar`, `fila` | Archivo .xlsx del reporte RF-14 | Solo ADMIN, solo agregados, sin fórmulas |
| Apache Commons Lang | commons-lang3 3.20.0 | Directa (T-58) | `ReporteExcelService.texto` | Detección de textos con riesgo de fórmula | `quotePrefixed` sin alterar el dato |
| Google Guava | guava 33.7.2-jre | Directa (T-58) | `ReporteExcelService.generar` | Invariante RF-14 y medición | No entrega archivos incoherentes |

BarberTurno / backend / src / main / java / pe / barberturno / reporting / ReporteExcelService.java ↑ Top

Code Blame 124 lines (117 loc) · 6 KB Raw

```java
public class ReporteExcelService {
    /**
     * Escribe las tres agrupaciones ya calculadas conservando literalmente los nombres.
     * @param actorId identidad del administrador para el registro técnico
     * @param resumen conteos conciliados de una misma fotografía RF-14
     * @param desde primer día aplicado en Lima
     * @param hasta último día aplicado en Lima
     * @param servicio nombre del servicio filtrado o Todos
     * @param profesional nombre del profesional filtrado o Todos
     * @return libro XLSX completo en memoria
     * @throws IOException si no puede serializar el libro
     * @throws IllegalStateException si las agrupaciones no suman el total
     */
    public byte[] generar(long actorId, ResumenReporteDto resumen, LocalDate desde, LocalDate hasta,
            String servicio, String profesional) throws IOException {
        try {
            Preconditions.checkState(
                    resumen.porEstado().values().stream().mapToLong(Long::longValue).sum() == resumen.total()
                    && resumen.porServicio().stream().mapToLong(ConteoReporteDto::total).sum() == resumen.total()
                    && resumen.porBarbero().stream().mapToLong(ConteoReporteDto::total).sum() == resumen.total(),
                    "Las agrupaciones del reporte no concilian.");
        } catch (IllegalStateException error) {
            LOG.warn("Exportación rechazada por invariante RF-14.");
            throw error;
        }
        var cronometro = Stopwatch.createStarted(ticker);
        try (var libro = new XSSFWorkbook(); var salida = new ByteArrayOutputStream()) {
            var hoja = libro.createSheet("Resumen");
            var textoSeguro = libro.createCellStyle();
            textoSeguro.setQuotePrefixed(true);
            fila(hoja, textoSeguro, "Reporte de reservas · BarberTurno", null);
            fila(hoja, textoSeguro, "Fecha de generación (Lima)", FECHA.format(TiempoNegocio.aLima(reloj.instant())));
            fila(hoja, textoSeguro, "Desde", desde == null ? "Inicio" : desde.toString());
            fila(hoja, textoSeguro, "Hasta", hasta == null ? "Hoy" : hasta.toString());
            fila(hoja, textoSeguro, "Servicio", servicio);
            fila(hoja, textoSeguro, "Profesional", profesional);
            fila(hoja, textoSeguro, "Total", resumen.total());
            fila(hoja, textoSeguro, "Por estado", null);
            for (var estado : EstadoReserva.values()) {
                fila(hoja, textoSeguro, etiqueta(estado), resumen.porEstado().getOrDefault(estado, 0L));
            }
            fila(hoja, textoSeguro, "Por servicio", null);
            for (var grupo : resumen.porServicio()) fila(hoja, textoSeguro, grupo.nombre(), grupo.total());
            fila(hoja, textoSeguro, "Por profesional", null);
            for (var grupo : resumen.porBarbero()) fila(hoja, textoSeguro, grupo.nombre(), grupo.total());
            hoja.setColumnWidth(0, 48 * 256);
            hoja.setColumnWidth(1, 48 * 256);
            libro.write(salida);
            byte[] archivo = salida.toByteArray();
            LOG.info("Reporte Excel generado: actor={} filas={} bytes={} duracionMs={}", actorId,
```

*Figura 9. ReporteExcelService publicado en GitHub (3bcd5d0): Guava Preconditions y Stopwatch, POI XSSFWorkbook y quotePrefixed, y el registro con Logback. Captura propia (21/10/2026).*

# 8. Interfaces gráficas implementadas

Todas las pantallas comprometidas en el APF2 están implementadas en Angular y funcionan contra la API real. La identidad visual «verde» (T-55 y T-57) se aplicó a todas ellas sin cambiar reglas, permisos ni contratos. Las capturas proceden del arnés de Playwright con datos ficticios y una base de pruebas aislada; cada una se tomó con análisis de accesibilidad axe sin violaciones y sin desplazamiento horizontal.

> Las capturas de cliente, barbero y administración proceden del arnés `frontend/e2e/capturas.spec.ts` sobre la base aislada `barberturno_test` con datos ficticios (`@ejemplo.test`). Todas se tomaron el 21/10/2026 sobre la versión entregable (`e0ebc54`, cuyo árbol es idéntico al de `3bcd5d0` en `main`) con el arnés en Chromium: 89 capturas, 46 de 46 casos aprobados y axe sin violaciones en cada una (evidencias/interfaces/indice.md).

| Rol | Pantallas y acciones disponibles |
|---|---|
| Público | Ingresar, crear cuenta (con consentimiento de privacidad), consultar servicios y disponibilidad |
| Cliente | Reservar en tres pasos; Mis citas con filtros; reprogramar y cancelar dentro de la política de 2 h; avisos internos; Mi cuenta y contraseña |
| Barbero | Agenda diaria y semanal propia; confirmar, iniciar, completar o marcar «no asistió»; ver la auditoría de sus reservas; avisos |
| Administrador | Agenda de todos; reservar en nombre de un cliente; servicios; barberos; horarios y bloqueos (también para todo el equipo); reportes con exportación a Excel; usuarios y restablecimiento de acceso; cambios excepcionales con motivo |

## 8.1 Cliente

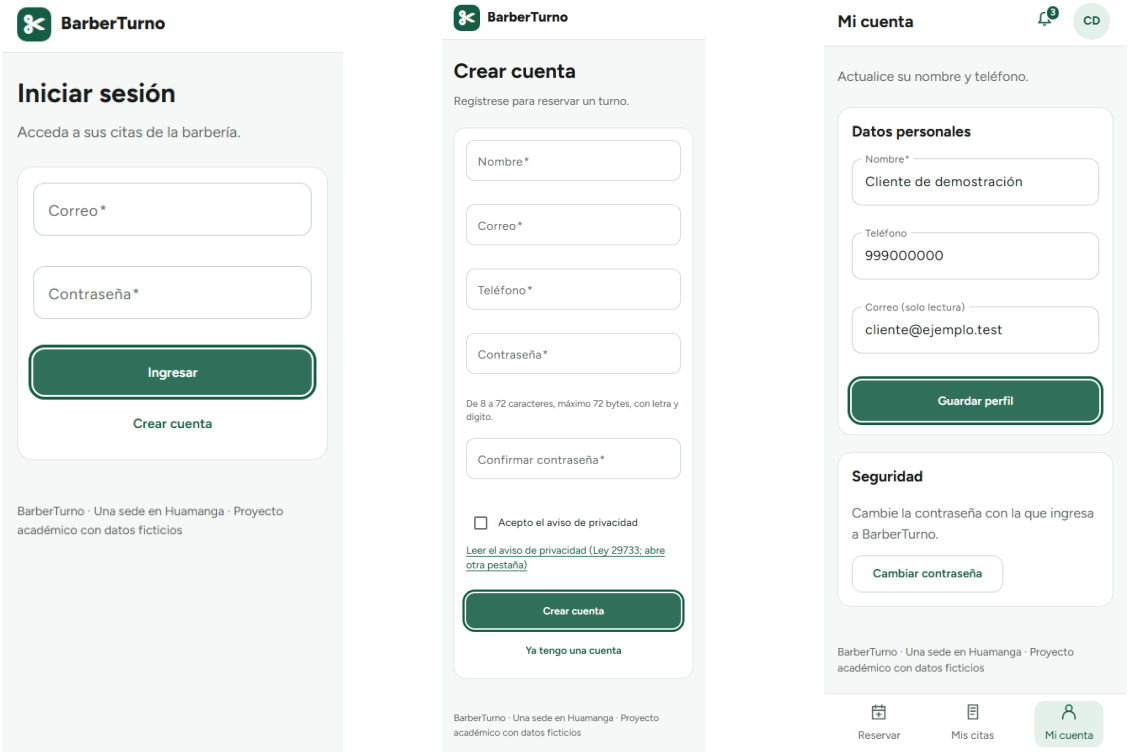



*Figura 10. Acceso en el teléfono (360 px): ingresar, crear cuenta con consentimiento y Mi cuenta.*

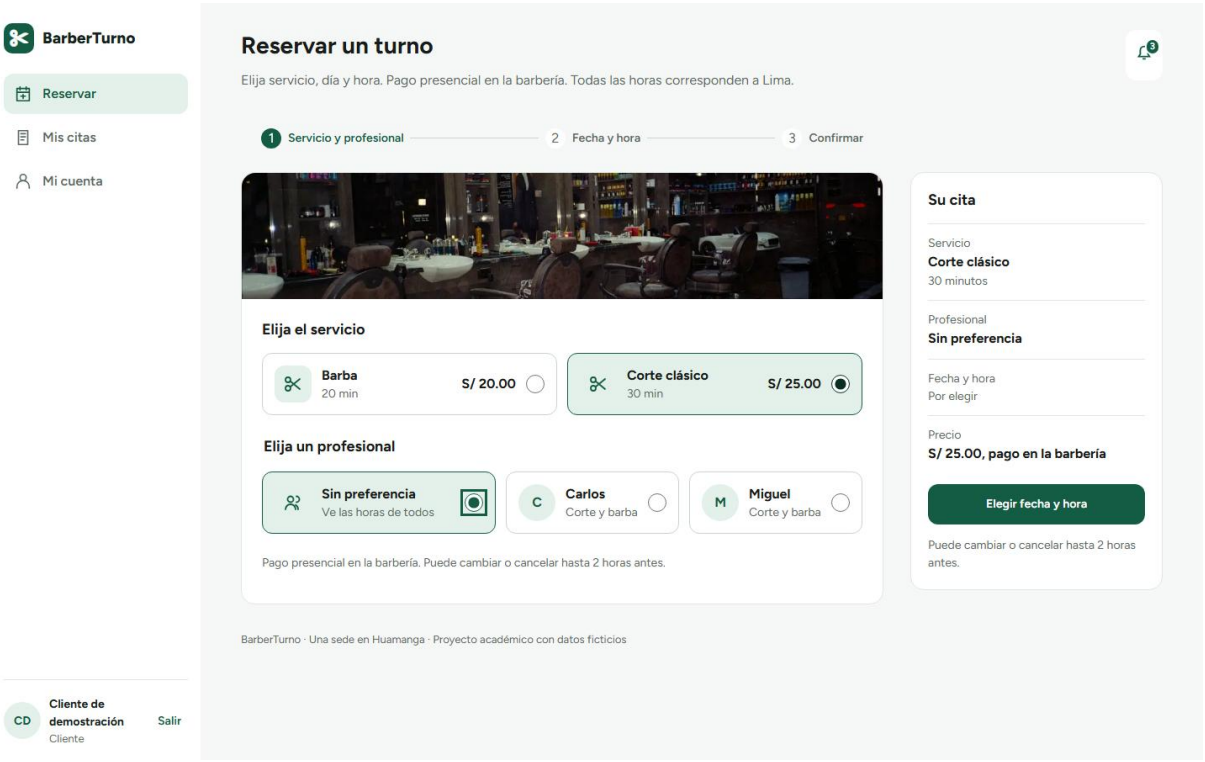



*Figura 11. Reserva guiada, paso 1 (1440 px): servicio, profesional, fecha y solo las franjas libres que calcula el servidor.*

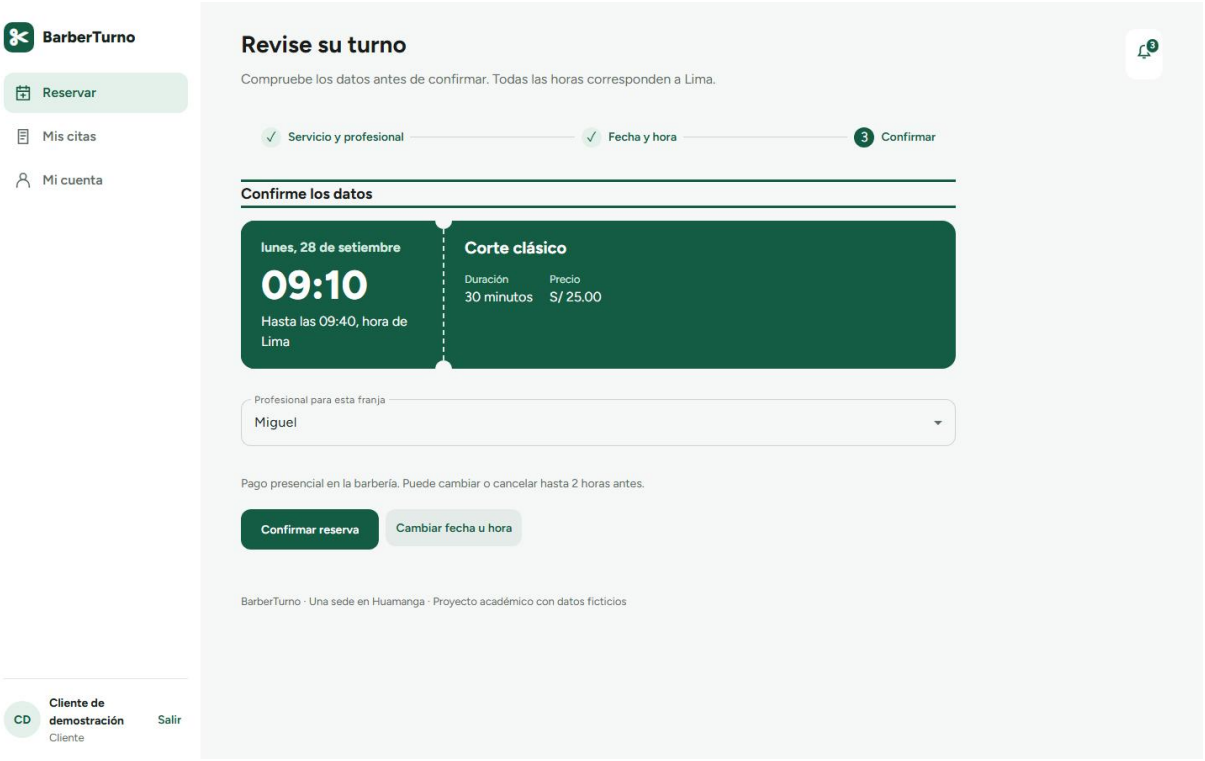



*Figura 12. Reserva guiada, paso 3: revisión con precio y duración antes de confirmar.*

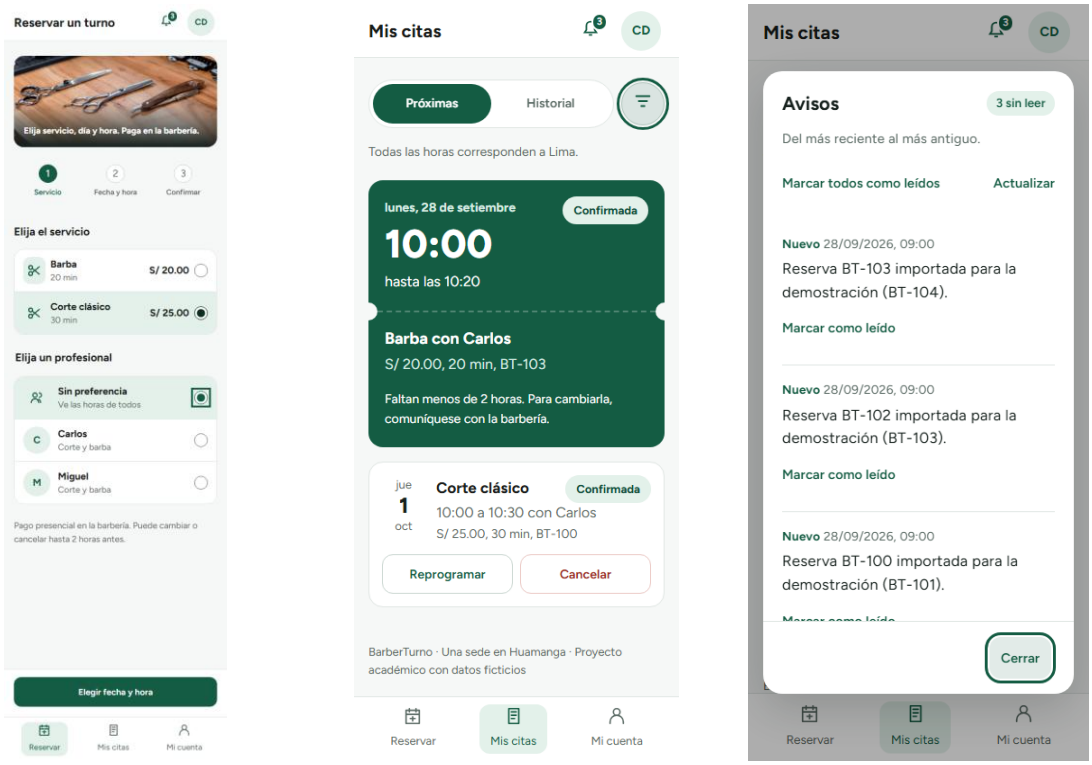



*Figura 13. En el teléfono (360 px): reservar, Mis citas y avisos internos.*

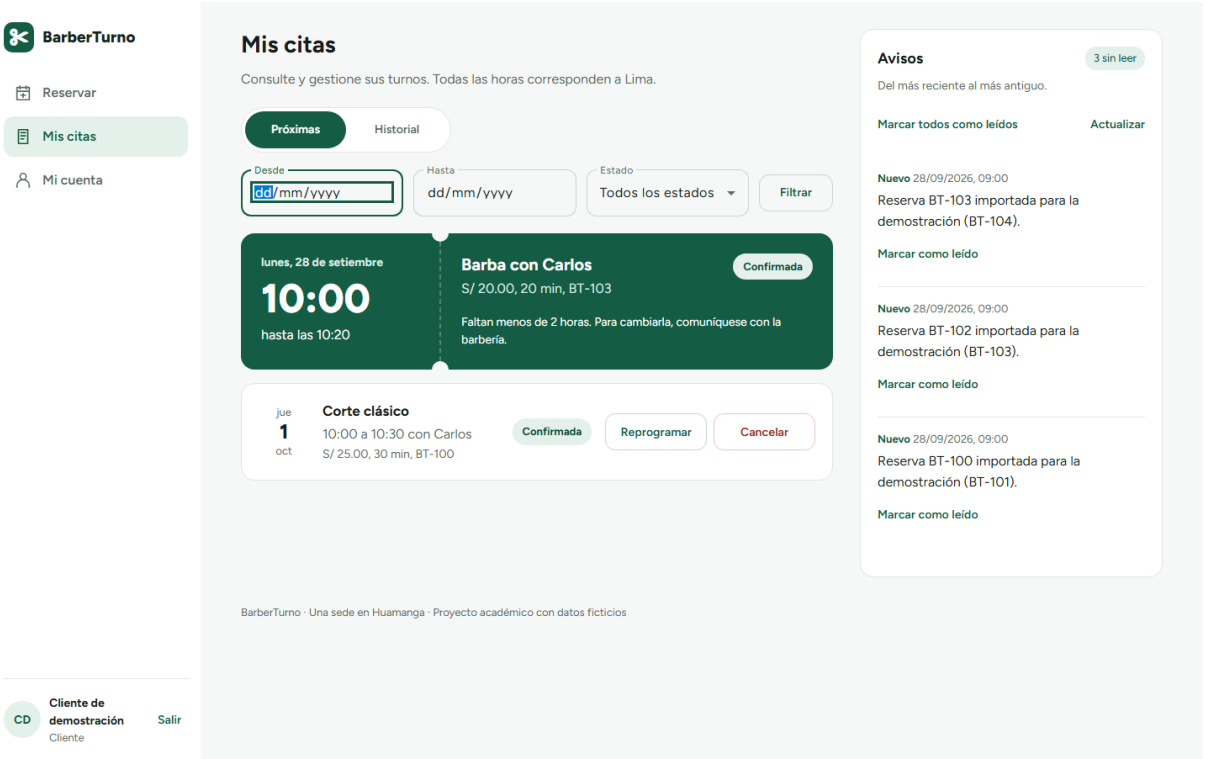



*Figura 14. Mis citas (1440 px): próximas e historial; los botones dependen de los permisos que envía el servidor.*

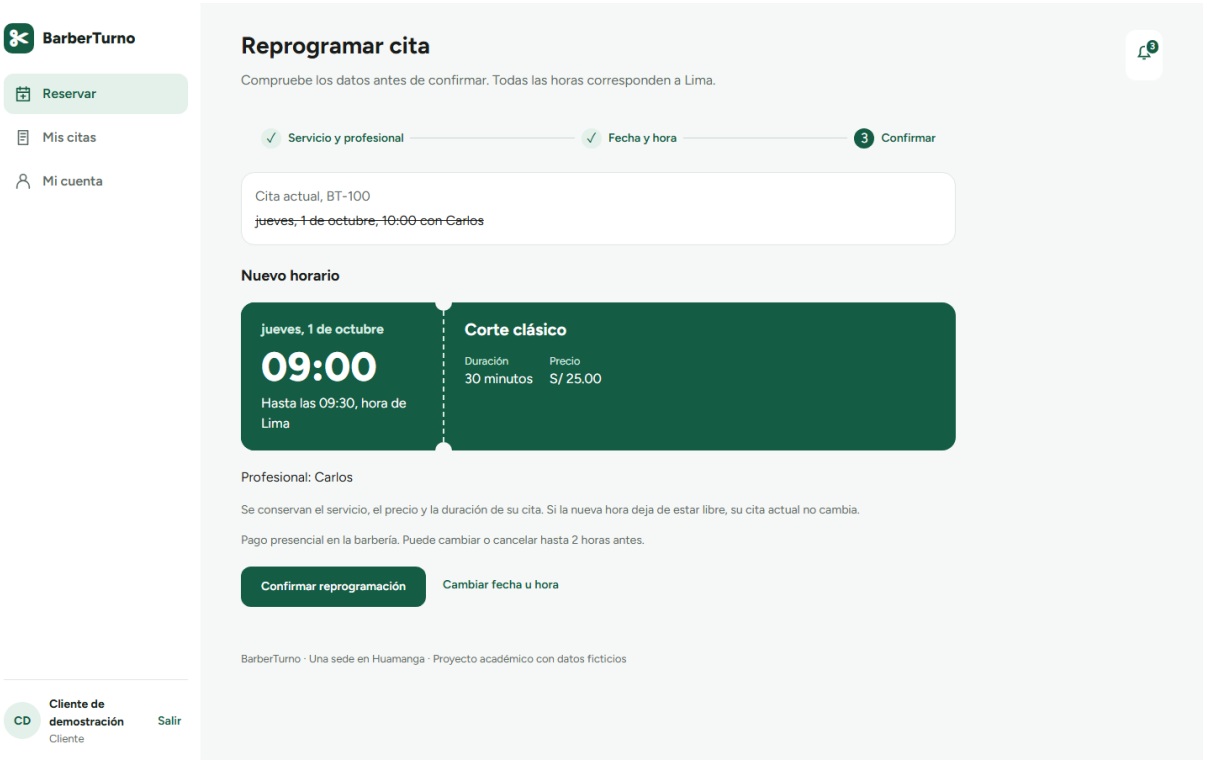



*Figura 15. Reprogramación por el cliente, permitida solo con dos horas o más de anticipación (RN-07).*

## 8.2 Barbero

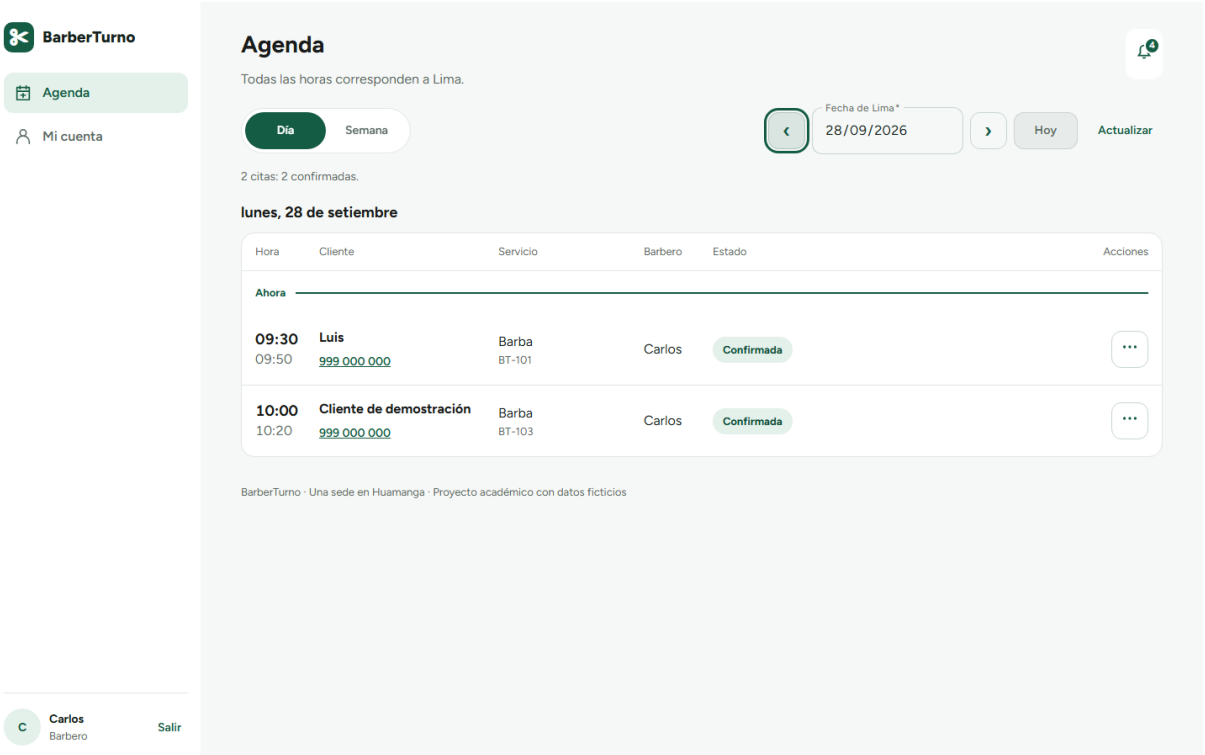



*Figura 16. Agenda diaria del barbero (1440 px): solo sus citas, con las acciones de atención permitidas.*

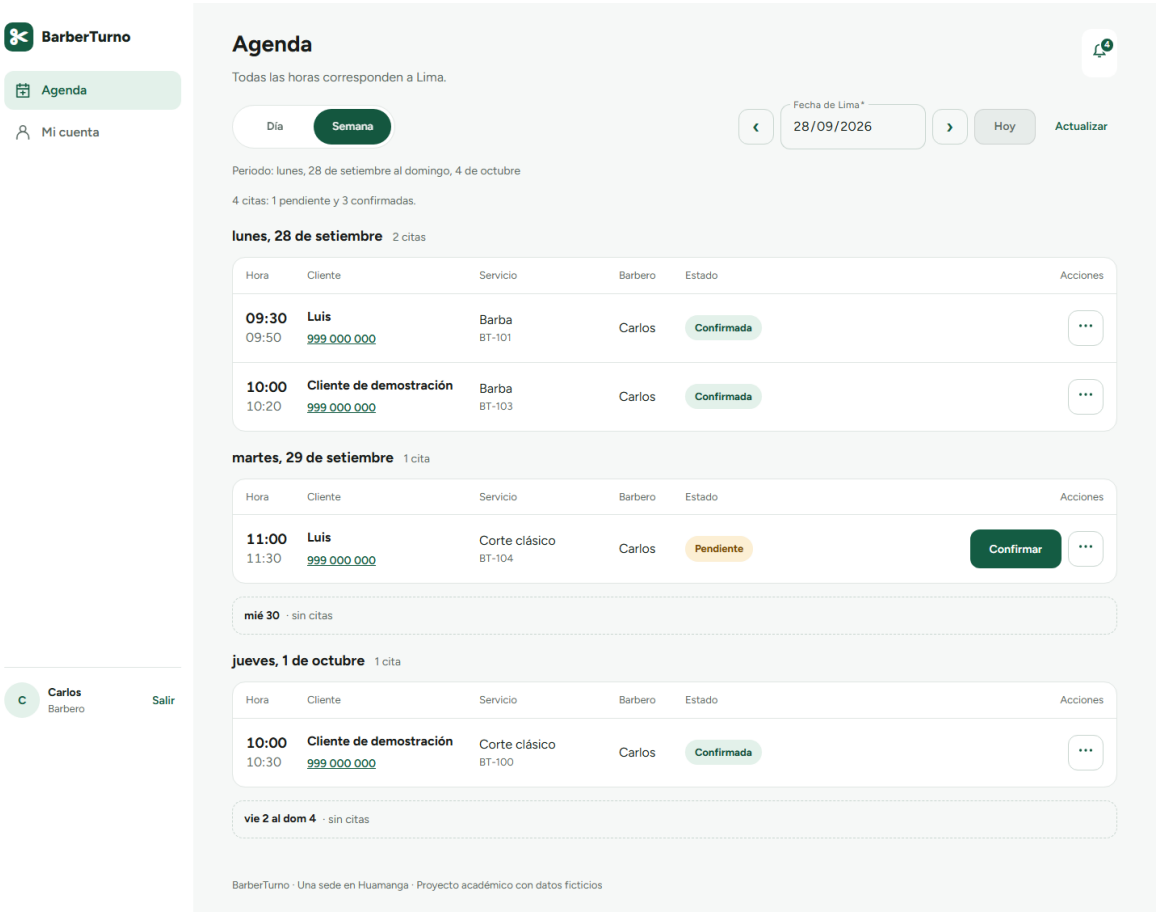



*Figura 17. Agenda semanal de lunes a domingo (1440 px).*

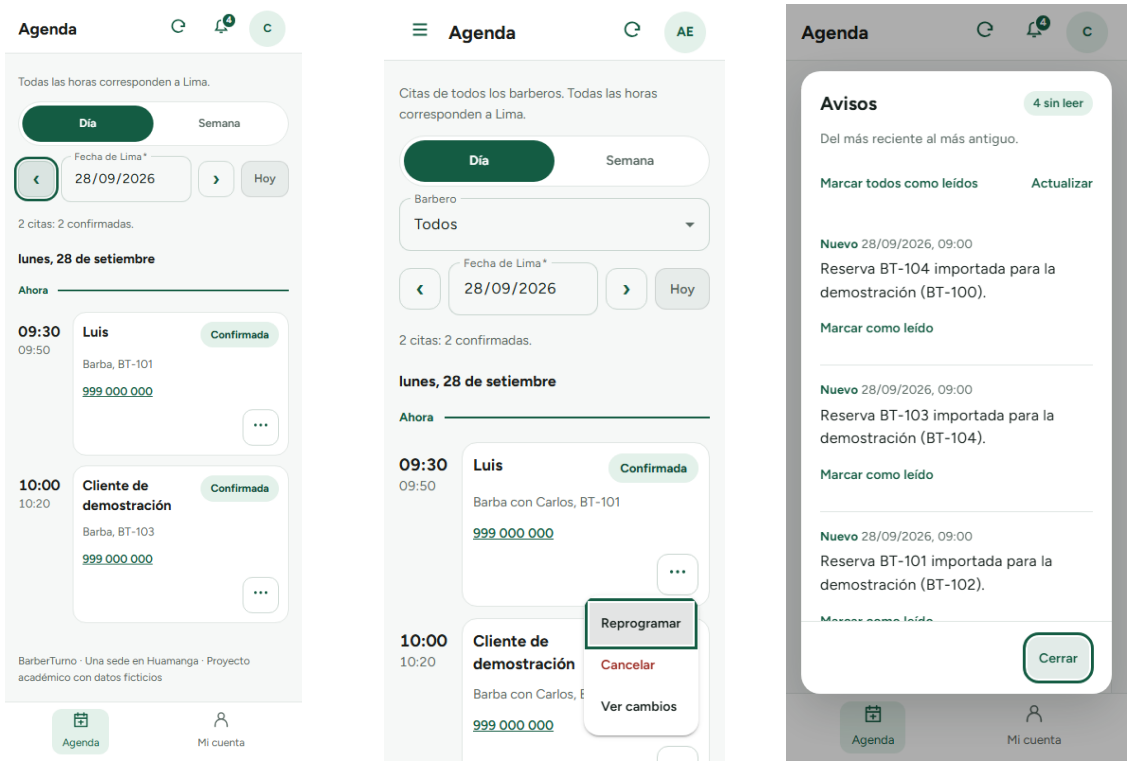



*Figura 18. Agenda en el teléfono, acciones de atención y avisos del barbero (360 px).*

## 8.3 Administrador

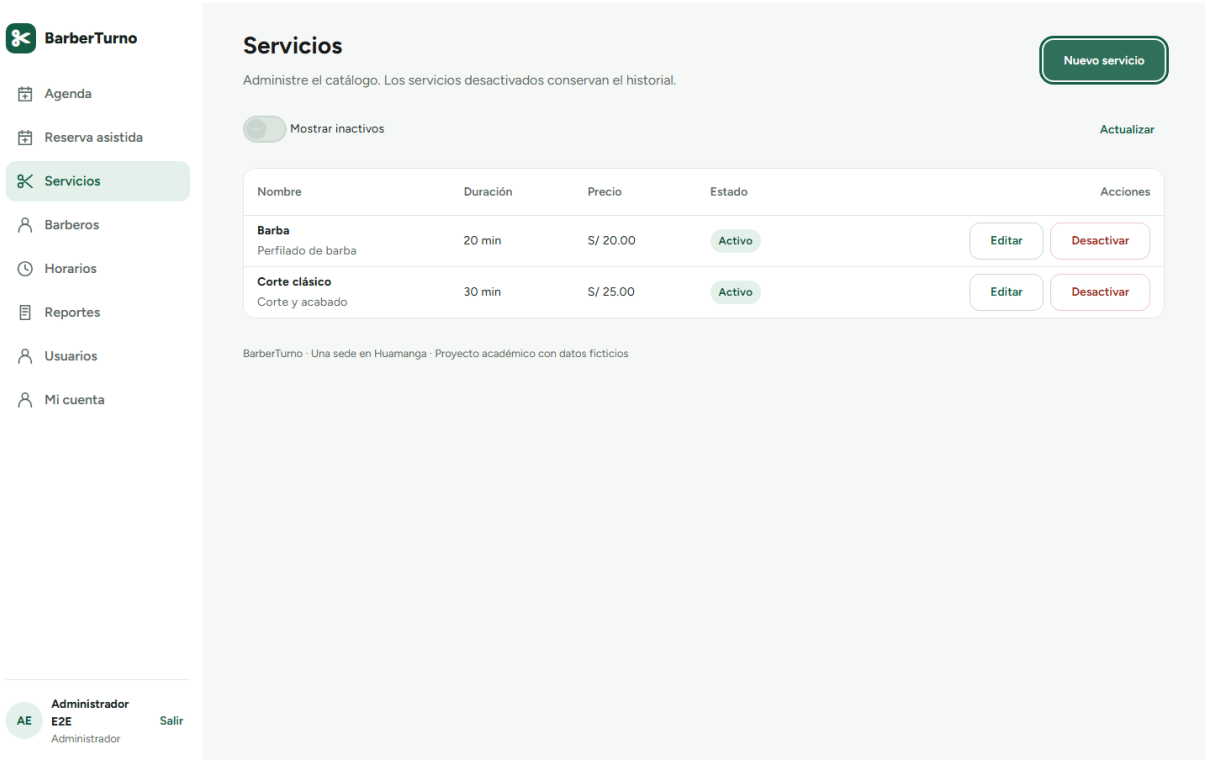



*Figura 19. Servicios (1440 px): alta, edición y desactivación; los inactivos no se ofrecen al reservar.*

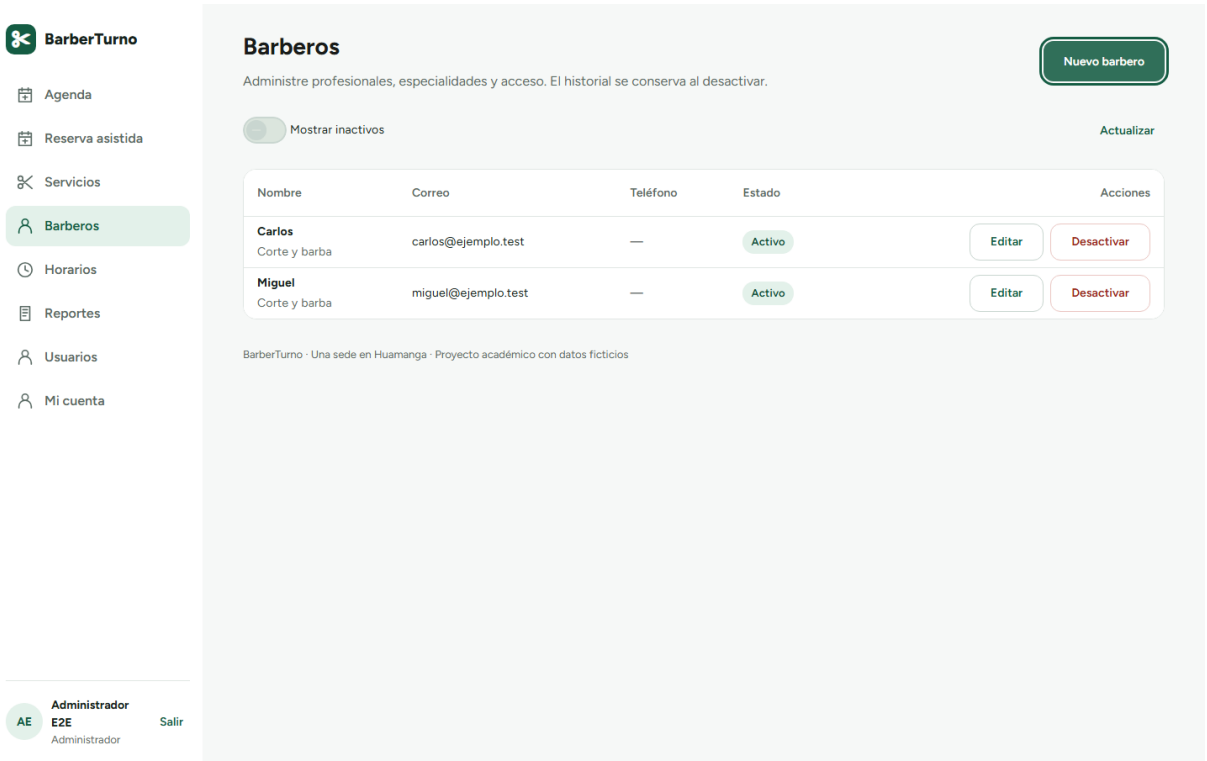



*Figura 20. Barberos: cuenta de personal con contraseña temporal y estado activo o inactivo (máximo diez activos).*

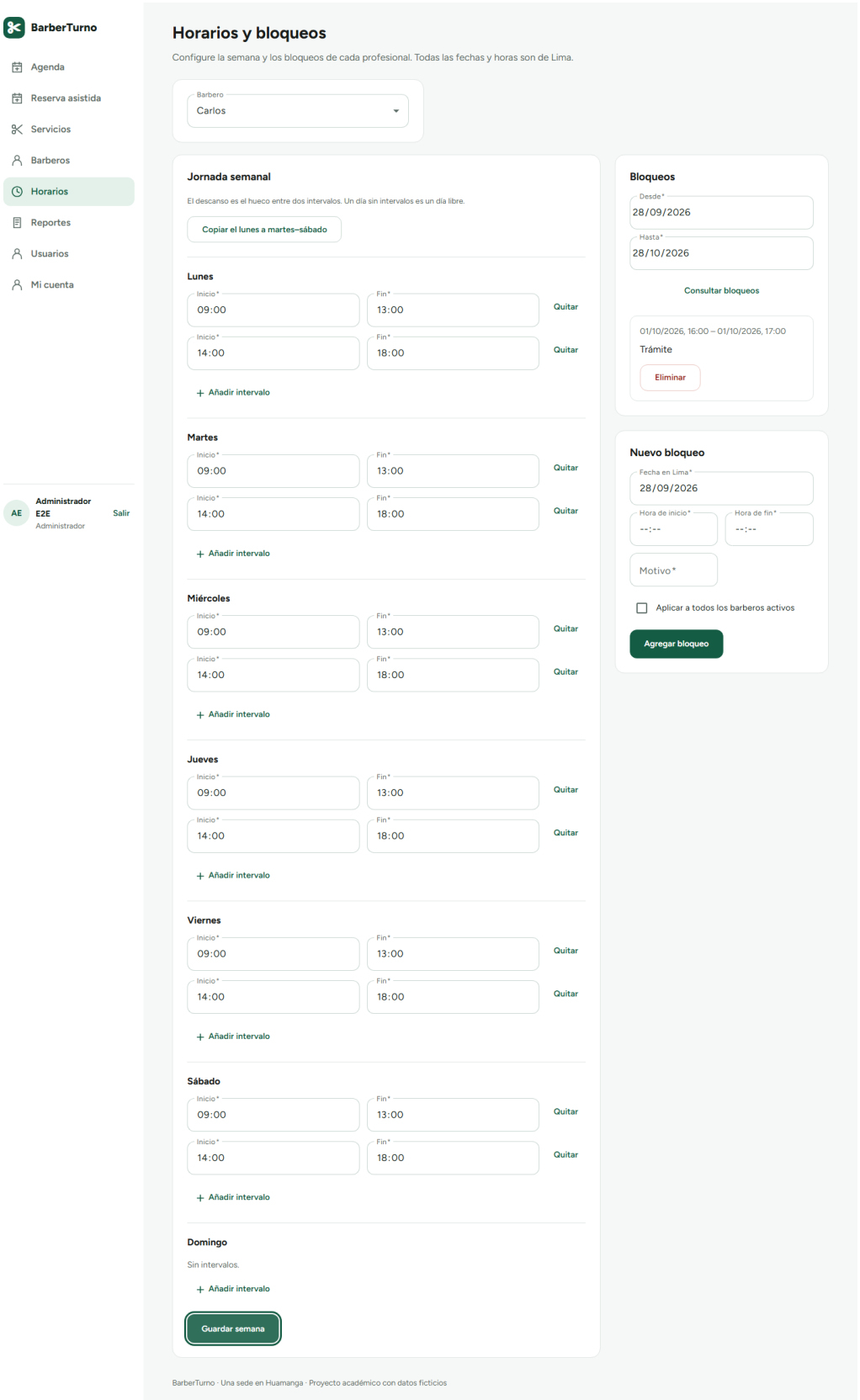



*Figura 21. Horarios y bloqueos de un barbero: jornada semanal por intervalos y bloqueos excepcionales.*

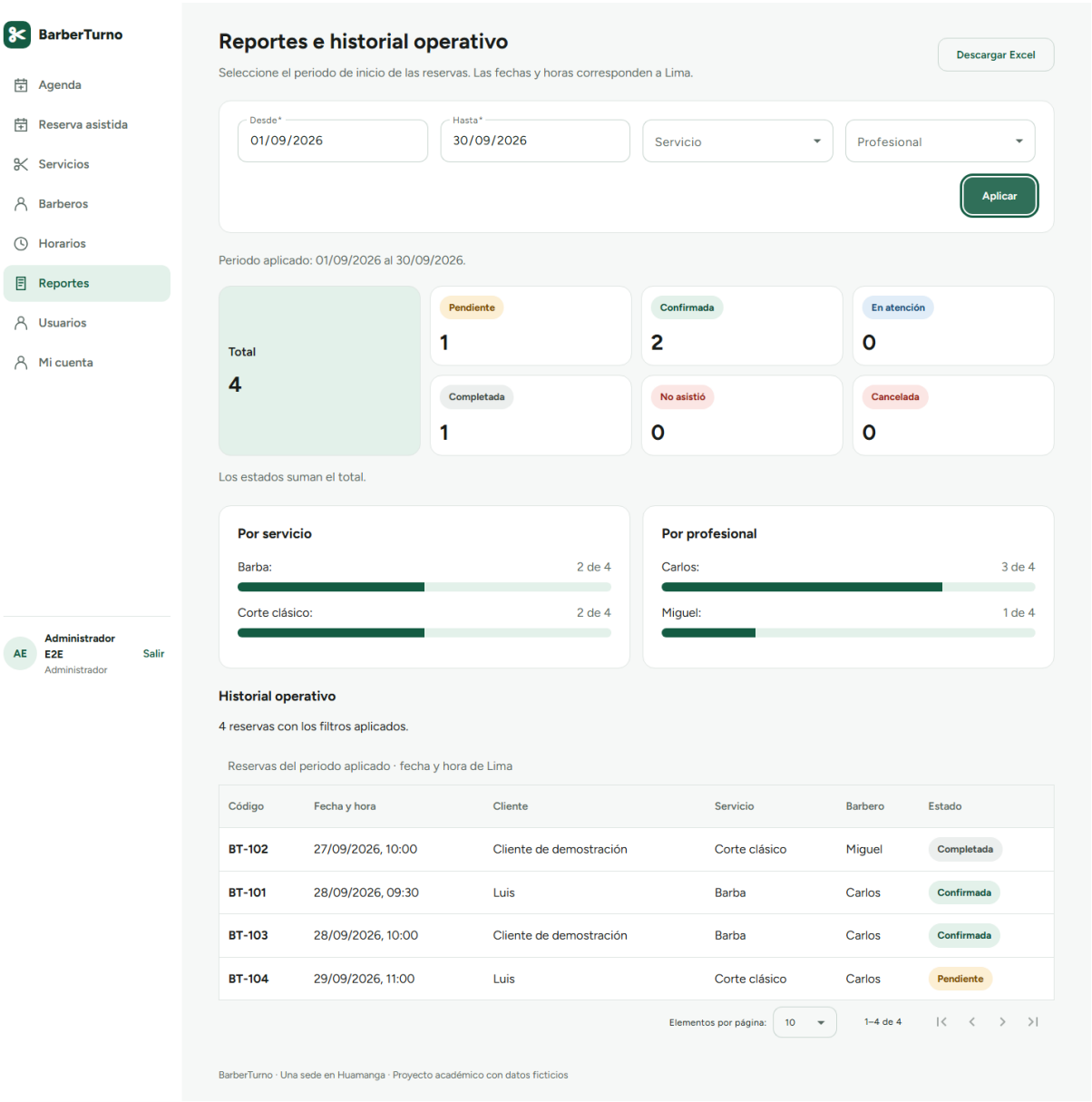



*Figura 22. Reportes (1440 px): conteos por estado, servicio y profesional, historial operativo con los mismos filtros y botón «Descargar Excel» (T-58).*

| | A | B |
|---|---|---|
| 1 | Reporte de reservas · BarberTurno | |
| 2 | Fecha de generación (Lima) | 28/09/2026 09:00:00 -05:00 |
| 3 | Desde | 2026-09-01 |
| 4 | Hasta | 2026-09-30 |
| 5 | Servicio | Todos |
| 6 | Profesional | Todos |
| 7 | Total | 4 |
| 8 | Por estado | |
| 9 | Pendiente | 1 |
| 10 | Confirmada | 2 |
| 11 | En atención | 0 |
| 12 | Completada | 1 |
| 13 | Cancelada | 0 |
| 14 | No asistió | 0 |
| 15 | Por servicio | |
| 16 | Barba | 2 |
| 17 | Corte clásico | 2 |
| 18 | Por profesional | |
| 19 | Carlos | 3 |
| 20 | Miguel | 1 |

*Figura 23. Libro descargado abierto en Microsoft Excel: hoja «Resumen» con filtros, total y conteos por estado, servicio y profesional; sin fórmulas.*

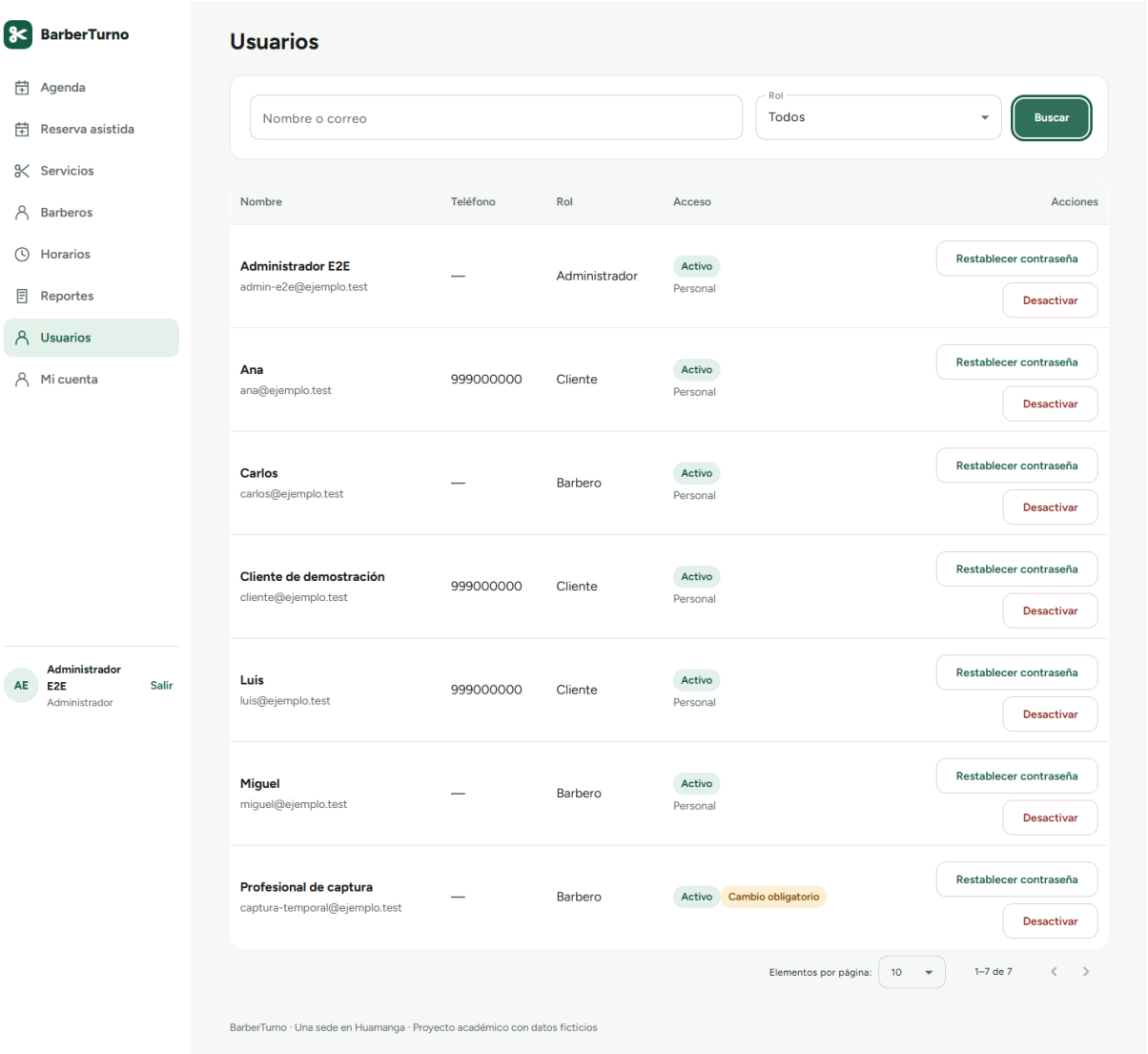



*Figura 24. Usuarios: búsqueda de clientes y personal y restablecimiento del acceso con contraseña temporal (RF-19).*

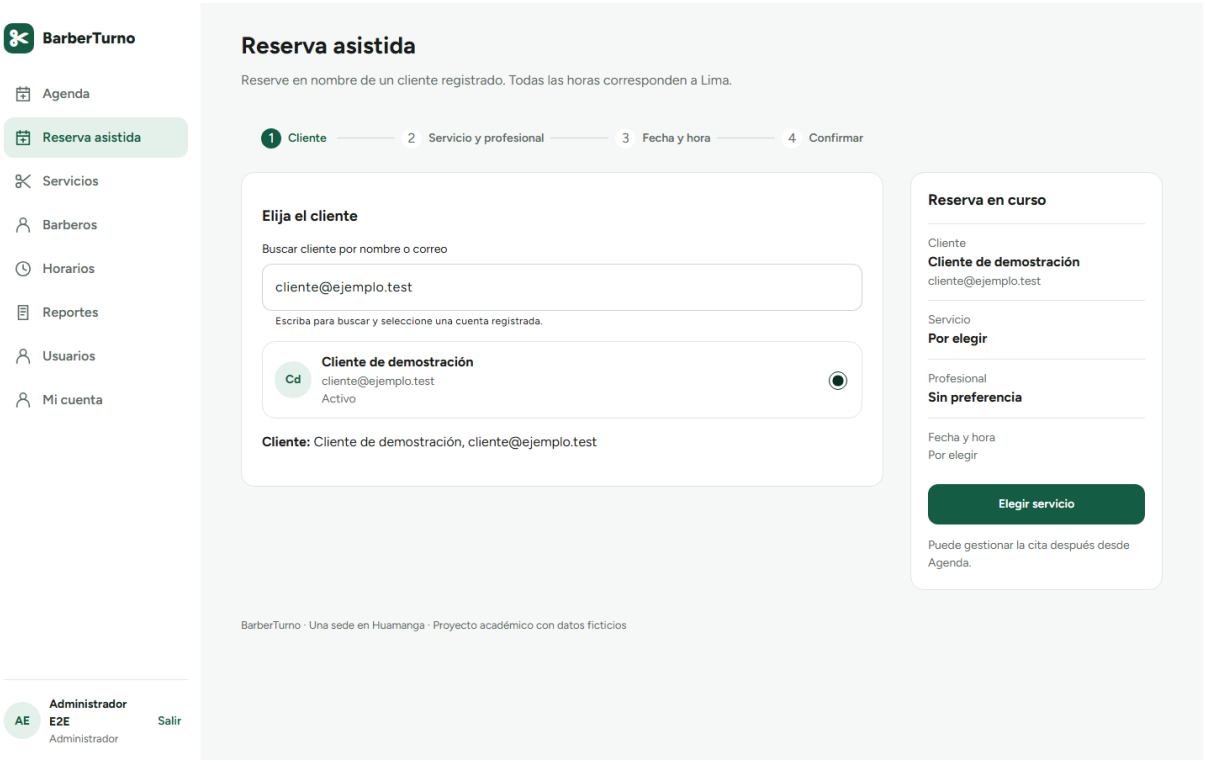



*Figura 25. Reserva asistida: el administrador reserva en nombre de un cliente registrado (RF-18).*

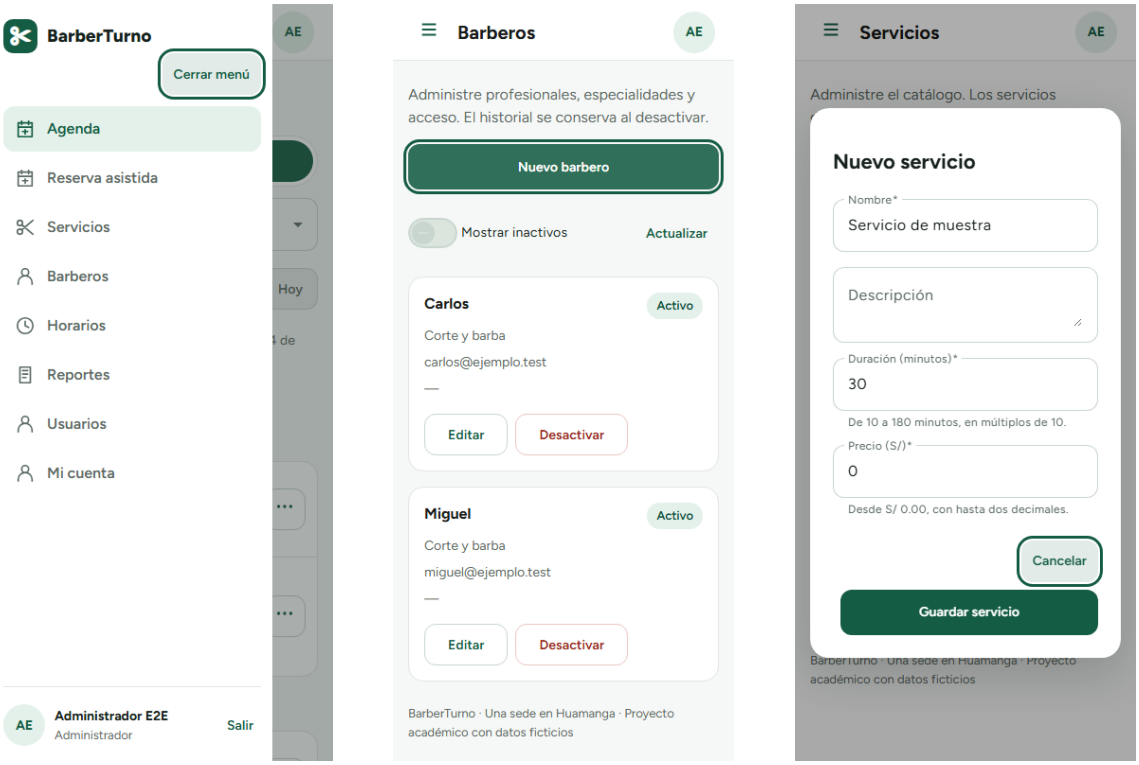



*Figura 26. Administración en 360 px: menú, barberos en tarjetas y diálogo de servicio.*

## 8.4 Estados de error, carga y acceso restringido

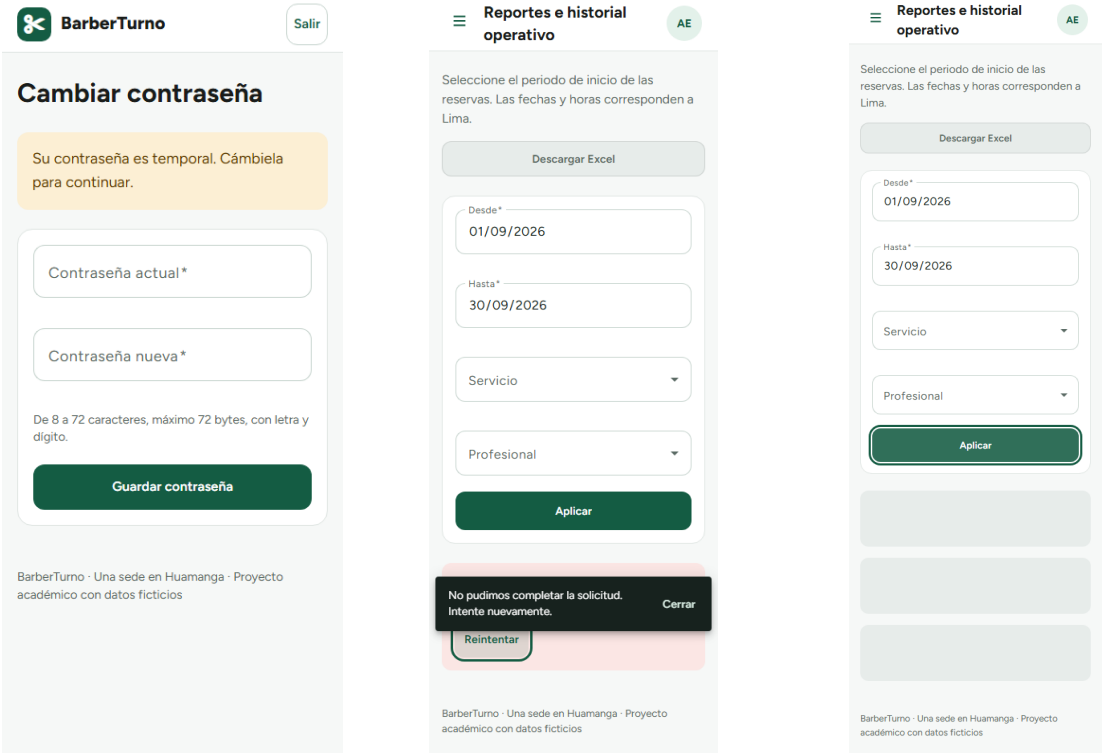



*Figura 27. Acceso restringido por rol, error homogéneo y estado de carga (360 px).*

# 9. Matriz requisito → pantalla → implementación → prueba

La matriz conecta cada requisito con su pantalla, el código que lo implementa y las pruebas que lo verifican. Las pruebas `*IT` usan PostgreSQL real; los casos E2E recorren la interfaz en Chromium y Firefox a 360 y 1440 px.

| Req. | Pantalla | Implementación (backend) | Pruebas |
|---|---|---|---|
| RF-01 Registro | /registro | `AuthController`, `AuthService.registrar` | CP-01 · AuthIT · E2E 07 |
| RF-02 Autenticación | /ingresar | `AuthService.login`, `SecurityConfig` | CP-02, CP-14, CP-17 · AuthIT · E2E 05 |
| RF-03 Perfil | /perfil | `PerfilController`, `PerfilService` | CP-01 · PerfilIT · E2E 07 |
| RF-04 Servicios | /admin/servicios | `ServicioController`, `ServicioService` | CP-08 · ServicioIT · E2E 04 |
| RF-05 Barberos | /admin/barberos | `BarberoController`, `BarberoService` | CP-08 · BarberoIT |
| RF-06 Horarios | /admin/horarios | `JornadaService`, `BloqueoService` | CP-07, CP-16 · JornadaIT, BloqueoIT · E2E 04 |
| RF-07 Disponibilidad | /reservar | `DisponibilidadService`, `CalculadoraFranjas` | CP-04 · DisponibilidadIT |
| RF-08 Reserva | /reservar | `ReservaService.crear` | CP-03, CP-04, CP-13, CP-15 · ReservaCrearIT, ConcurrenciaReservaIT · E2E 01/06 |
| RF-09 Reprogramación | /mis-citas, /agenda | `ReservaService.reprogramar` | CP-05 · ReprogramacionIT · E2E 01 |
| RF-10 Cancelación | /mis-citas, /agenda | `ReservaService.cancelar` | CP-06 · CancelacionIT · E2E 01/02 |
| RF-11 Agenda | /agenda | `ReservaConsultaService` | CP-02 · ReservaConsultaIT · E2E 03 |
| RF-12 Atención | /agenda | `ReservaService.transicionar`, `PoliticaTransiciones` | CP-09 · TransicionesIT · E2E 03 |
| RF-13 Historial | /mis-citas, /admin/reportes | `ReservaConsultaService`, `ReservaFiltros` | CP-02, CP-10 · ReservaConsultaIT, ReporteIT |
| RF-14 Reportes | /admin/reportes | `ReporteService`, `ReporteRepository`; `ReporteExcelService` | CP-10, CP-20 · ReporteIT, ReporteExcelServiceTest (12), ReporteExcelIT (18) · E2E 04 |
| RF-15 Contraseña | /cambiar-password | `AuthService`, `PasswordTemporalFilter` | CP-17 · PerfilIT |
| RF-16 Avisos | campana, /mis-citas | `NotificacionService` | CP-18 · AuditoriaAvisosIT · E2E 01 |
| RF-17 Auditoría | /agenda | `AuditoriaService` | CP-09 · AuditoriaAvisosIT |
| RF-18 Reserva asistida | /reservar (ADMIN) | `ReservaService.crear` con `clienteId` | CP-19 · ReservaAsistidaIT |
| RF-19 Restablecer acceso | /admin/usuarios | `UsuarioAdminService` | CP-17 · UsuarioAdminIT |
| RF-20 Bloqueo del equipo | /admin/horarios | `BloqueoService` (lote) | CP-07 · BloqueoIT |
| RF-21 Sin preferencia | /reservar | `DisponibilidadService` | DisponibilidadIT |

Fuente: `docs/requisitos.md` §9 y `docs/pruebas/aceptacion.md` (matriz completa con los métodos de prueba). Los RNF se tratan en la sección 13.

# 10. Control de versiones con Git y GitHub

El proyecto se versiona con Git desde su primer commit (01/10/2026) y se publica en el repositorio público [github.com/sohnny-lima/BarberTurno](github.com/sohnny-lima/BarberTurno). A la fecha del informe, `main` en GitHub tiene **450 commits** y 55 merges de tareas, y cada push ejecuta la integración continua.

| Práctica | Cómo se aplicó | Evidencia |
|---|---|---|
| Una rama por tarea | `tarea/T-XX-descripcion` desde `main`; integración con `git merge --no-ff` para conservar la historia de cada tarea | 54 ramas de tarea locales (su historia llega a GitHub mediante los merges de `main`) ramas de tarea; merges en `main` |
| Commits convencionales en español | `feat(reservas): …`, `fix(auth): …`, `test(e2e): …`, `docs: …`, con la referencia `[T-XX]` | feat 83, fix 55, test 59, docs 223, chore/ci 18 |
| Integración continua | GitHub Actions `ci.yml`: backend `mvnw verify` con PostgreSQL 18 y frontend lint, formato, Vitest y build | Primer CI verde: ejecución 37314699514 (05/10/2026); CI de `d2f3107`: 39099141093 (success) |
| Trazabilidad | Cada tarea registra fecha, hashes y notas de cierre en `docs/tareas.md`; cada requisito apunta a sus tareas y pruebas | `docs/tareas.md`, `docs/requisitos.md` §9 |
| Protección de secretos | `.gitignore` excluye `.local/`, `.env*`, claves, `target/`, `node_modules/` | Revisado antes de cada publicación |

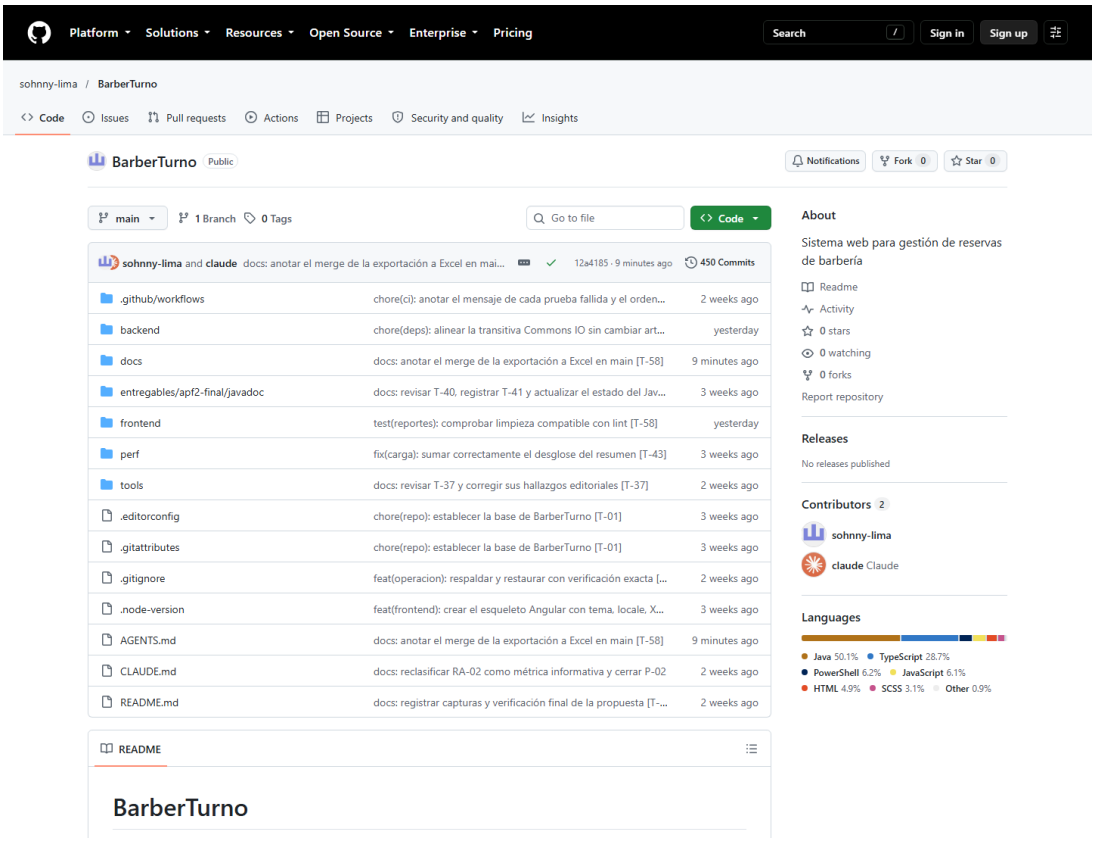



*Figura 28. Repositorio público en GitHub con 450 commits y CI correcta en el último commit. Captura propia (21/10/2026).*

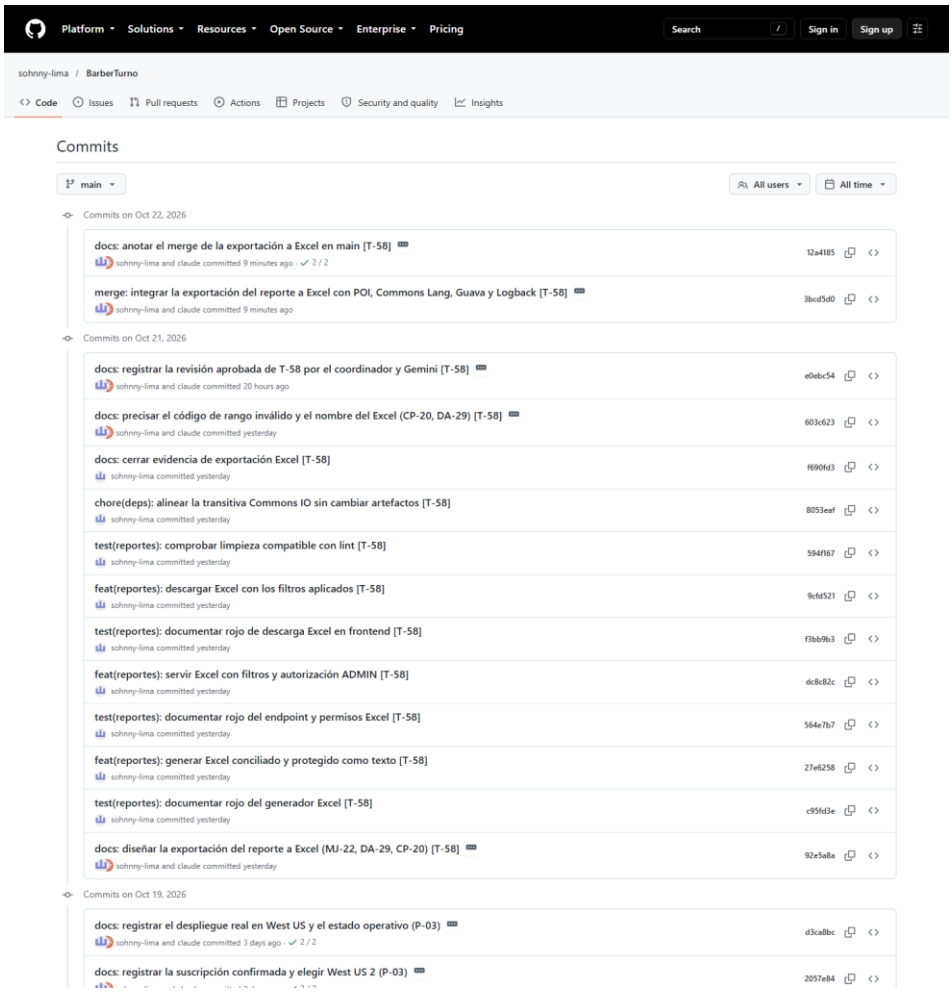



*Figura 29. Historial de commits de main en GitHub (últimos avances del cierre y de la fase 2). Captura propia (21/10/2026).*

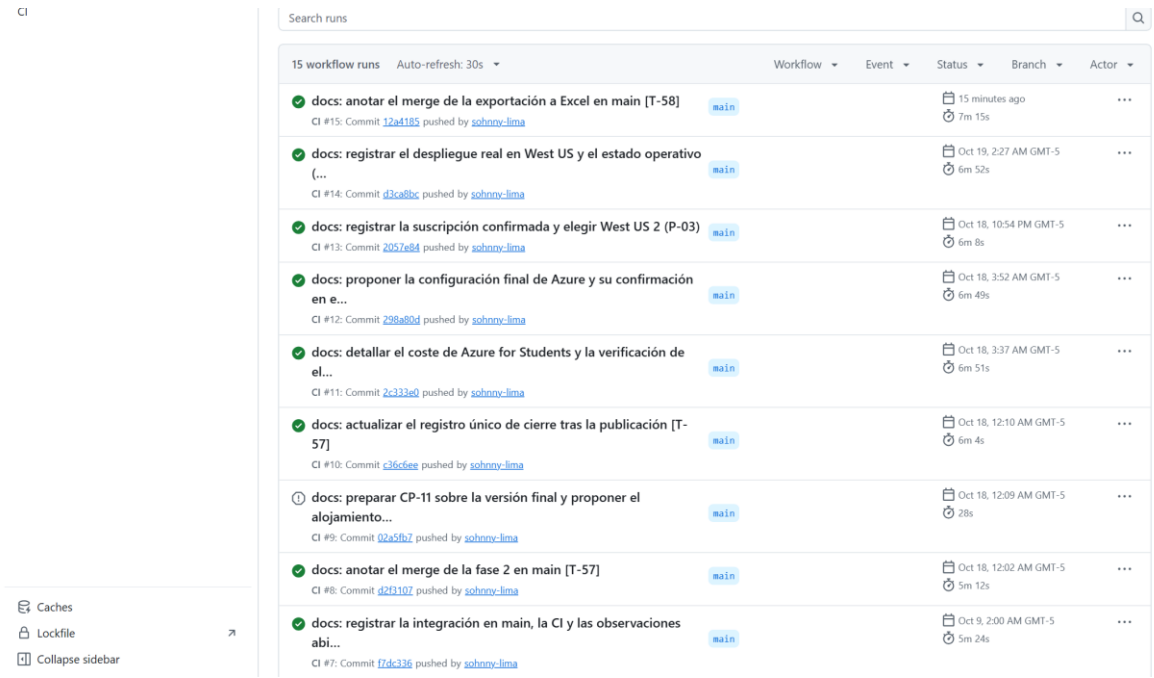



*Figura 30. GitHub Actions: ejecuciones del flujo CI sobre main. Captura propia (21/10/2026).*

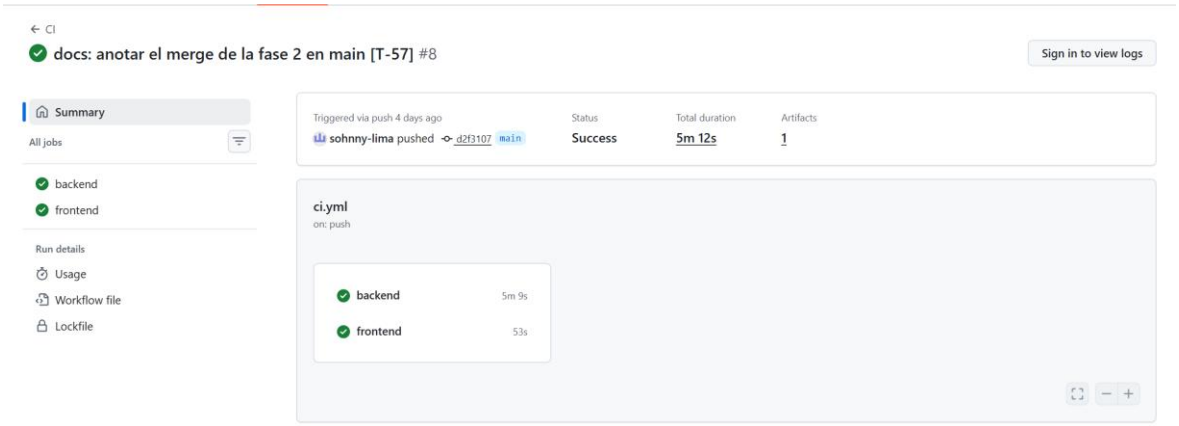



*Figura 31. Ejecución 39099141093 sobre d2f3107: jobs backend y frontend correctos. Captura propia.*

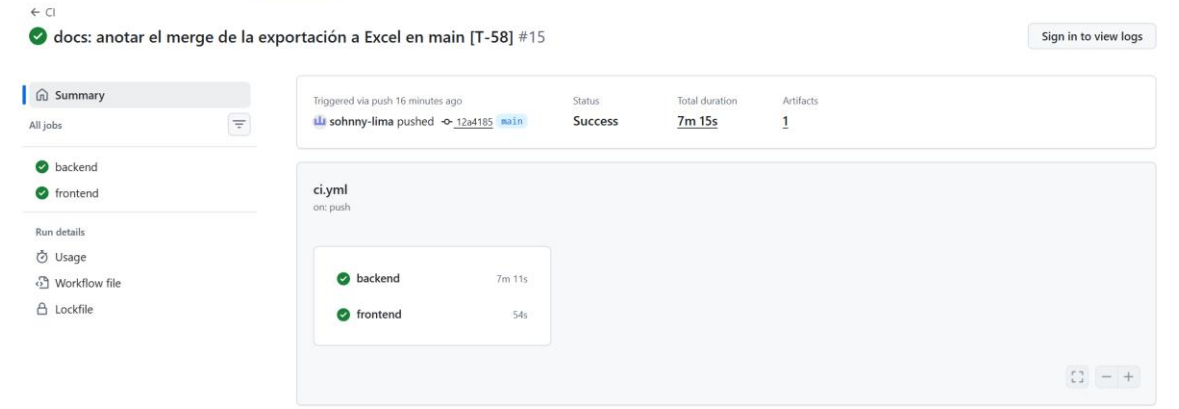



*Figura 32. Ejecución 39716850704 tras integrar T-58 (12a4185, código de 3bcd5d0): jobs backend y frontend correctos. Captura propia (21/10/2026).*




*Figura 33. Merge sin avance rápido de T-58 en main (3bcd5d0), con sus dos padres. Captura propia (21/10/2026).*

## 10.1 Evolución por fases

| Periodo | Tareas | Avance |
|---|---|---|
| 01/10/2026 | T-01…T-09 | Repositorio, esqueletos, CI, medición de Java, esquema Flyway, entidades y reglas de dominio |
| 02–04/10/2026 | T-10…T-37 | Identidad, catálogo, horarios, disponibilidad, reservas con concurrencia, agenda, reportes, producción, E2E, carga y recuperación |
| 04–06/10/2026 | T-38…T-49 | Ajustes de revisión, Javadoc completo, primer CI remoto (T-47), corrección CSRF de Firefox (T-49) |
| 05–09/10/2026 | T-48…T-56 | Interfaz con Tailwind y dirección visual «verde» (fase 1) |
| 17–19/10/2026 | T-57, P-03 | Fase 2 visual, checkpoint, despliegue en Azure |

| Periodo | Tareas | Avance |
|---|---|---|
| 21/10/2026 | T-58 | Exportación a Excel con POI, Commons Lang, Guava y Logback, con TDD; integrada en `main` (merge `3bcd5d0`) y publicada |

## 10.2 Qué está publicado y qué no

> **Publicado en GitHub (21/10/2026, autorizado por el responsable):** el código de T-58 integrado en `main` (merge `3bcd5d0`; CI 39716850704 success); los materiales del APF3 en `entregables/apf3/` (sin los ZIP, que se entregan en el aula); y el informe final del APF2, sin modificar, en `entregables/apf2-final/`. Los materiales se publicaron en un commit de documentación posterior al del código, registrado en `docs/cierre.md`. El borrador del APF2 sigue en `docs/apf2/` como referencia de solo lectura. Las capturas de GitHub de este informe se tomaron después de publicar el código.

# 11. Pruebas y resultados técnicos

Los resultados se identifican con su commit, fecha y entorno. Las cifras del checkpoint no se presentan como ejecuciones nuevas.

| Nivel | Herramienta | Resultado | Procedencia |
|---|---|---|---|
| Backend (unitarias + integración con PostgreSQL) | JUnit 6, AssertJ, MockMvc | 1548/1548 | Checkpoint del coordinador 17–18/10/2026, `mvnw clean verify`, código = `d2f3107` |
| Frontend | Vitest | 442/442 en Lima y en Madrid | Mismo checkpoint |
| E2E | Playwright + axe | 48/48 (Chromium y Firefox, 360 y 1440 px) | Mismo checkpoint |
| Build de producción | Angular CLI | Inicial 496,39 kB (presupuesto 500 kB, sin elevar) | Mismo checkpoint |
| CI remota | GitHub Actions | success (39099141093) y success (39716850704) | `d2f3107`, 18/10/2026; `12a4185` (código de `3bcd5d0`, T-58), 21/10/2026 |
| Cobertura de líneas | JaCoCo (umbral ≥ 70 % en `reservations` y `scheduling`, obligatorio en `verify`) | Último registro publicado: backend 98,94 %, reservations 100 %, scheduling 99,50 % | `docs/pruebas/aceptacion.md` (T-37, 05/10/2026); el umbral se vuelve a exigir en cada `verify` |
| Carga (RNF-01) | Gatling | 50 usuarios concurrentes; p95 ≤ 2 s en disponibilidad | T-35/T-43, equipo local (no producción) |
| Recuperación (RNF-09) | `tools/respaldo.sh`, `restaurar.sh` | Ensayo local de respaldo y restauración verificado | T-36 (local; en Azure, pendiente) |
| T-58 · backend | `mvnw clean verify` | 1578/1578 (+30: 12 del generador y 18 del endpoint); JaCoCo `reporting` 100 %; Javadoc sin avisos | Codex, 21/10/2026, rama de T-58 (`8053eaf`); el coordinador repitió las 60 pruebas de reportes |
| T-58 · frontend | Vitest, Angular CLI | 450/450 en Lima y Madrid; build inicial 496,42 kB (+0,03 kB) | Codex, 21/10/2026 |
| T-58 · E2E | Playwright | 48/48 con la descarga en Chromium y Firefox | Codex, 21/10/2026 |
| T-58 · TDD | Commit rojo en un worktree | `c95fd3e`: 12 de 12 fallan por aserción | Repetido por el coordinador, 21/10/2026 |

## 11.1 Participación de Java frente al compromiso del APF2

El APF2 estimó para A1 una participación de Java del **60 %** y definió cómo verificarla (sección 6 del APF2): líneas de código fuente propias en Java divididas entre todas las líneas de código fuente del producto, sin dependencias, archivos generados, comentarios, líneas vacías ni documentación, informando el resultado con y sin pruebas. El repositorio aplica exactamente ese método con `node tools/medir-java.mjs`, que cuenta `.java`, `.ts`, `.html`, `.scss`, `.css` y `.sql` en `backend/src`, `frontend/src`, `frontend/e2e` y `perf`, y deja cada medición fechada en `docs/pruebas/medicion-java.md`.

| Fecha (Lima) | Commit | Momento del proyecto | Java sin pruebas | Java con pruebas |
|---|---|---|---|---|
| 01/10/2026 | `919e9ca` | Esqueletos del backend y del frontend | 51,47 % | 72,15 % |
| 04/10/2026 | `8b22a16` | Dominio, API y seguridad avanzados; poca interfaz | 61,76 % | 75,75 % |

| Fecha (Lima) | Commit | Momento del proyecto | Java sin pruebas | Java con pruebas |
|---|---|---|---|---|
| 04/10/2026 | `5ac8d01` | Interfaces de reserva, agenda y reportes | 42,79 % | 58,79 % |
| 04/10/2026 | `94dbe99` | Cierre de T-01…T-47 | 40,25 % | 55,02 % |
| 21/10/2026 | `12a4185` | Tras las fases visuales (T-48, T-55, T-57) y T-58; versión entregada | 28,78 % | 41,56 % |

| Líneas de producto | Java | TypeScript | HTML | SCSS/CSS | SQL | Total |
|---|---|---|---|---|---|---|
| 05/10/2026 (94dbe99) | 4284 | 3978 | 1704 | 568 | 109 | 10 643 |
| 21/10/2026 (12a4185) | 4444 | 5032 | 2675 | 3180 | 109 | 15 440 |
| Variación | +160 | +1054 | +971 | +2612 | 0 | +4797 |

**Qué muestra la medición.** La brecha frente al 60 % es real: con el método comprometido, Java representa el 28,78 % del código de producto y el 41,56 % contando las pruebas. La caída no se debe a que la lógica haya salido de Java: entre el 05/10 y el 21/10 el código Java creció (+160 líneas, sobre todo T-58), mientras que la capa de presentación creció en 4637 líneas, en su mayor parte hojas de estilo (+2612 líneas de SCSS/CSS) y plantillas de las fases visuales aprobadas. Las reglas de negocio, la autorización, las transacciones, la persistencia y los reportes siguen íntegramente en Java; el frontend no contiene reglas (sección 4).

**Qué implica.** (1) El 60 % del APF2 era una estimación hecha antes de diseñar la interfaz definitiva y no se cumple; se informa tal cual. (2) El responsable reclasificó el porcentaje como métrica informativa el 04/10/2026 (RA-02, P-02 en `docs/requisitos.md`); el requisito que sí se mantiene es que el backend y toda la lógica estén en Java, y se cumple. Esa reclasificación es una decisión del proyecto: si el docente evalúa el porcentaje, la brecha debe defenderse con estos datos. (3) No se cambió la métrica ni se añadió código para inflarla: hacerlo iría contra la claridad del diseño (DA-15). (4) GitHub muestra Java 50,1 %, pero mide bytes por archivo con otro criterio (Linguist); no sustituye al método comprometido y no se usa como evidencia.

# 12. Despliegue en Azure y monitorización

El responsable creó los recursos y desplegó manualmente el 19/10/2026 en **Azure for Students**, región West US. La aplicación se publica como un único jar construido con el perfil `con-frontend`, de modo que la SPA y la API comparten origen, sin CORS.

| Recurso | Configuración |
|---|---|
| App Service `barberturno-web-202610` | Linux B1, Java SE 21, *Always On*, *HTTPS Only*, *Health check* en `/actuator/health` |
| PostgreSQL `pg-barberturno-202610` | Flexible Server 18.6 B1ms, 32 GB, copias automáticas con retención de 7 días; base `barberturno_prod` con rol de aplicación propio y `btree_gist`; conexión TLS `verify-full` |
| Artefacto | Jar con SHA-256 `aa4dcf8f…88bc2`, código idéntico a `d2f3107`; despliegue `b850dc93…` con resultado `RuntimeSuccessful` |
| Monitor externo | UptimeRobot, `GET /actuator/health` cada 5 minutos con alertas por correo |
| Coste | ≈ 13,14 USD/mes del crédito de 100 USD (App Service); PostgreSQL B1ms dentro del beneficio gratuito, pendiente de confirmar en *Usage* |

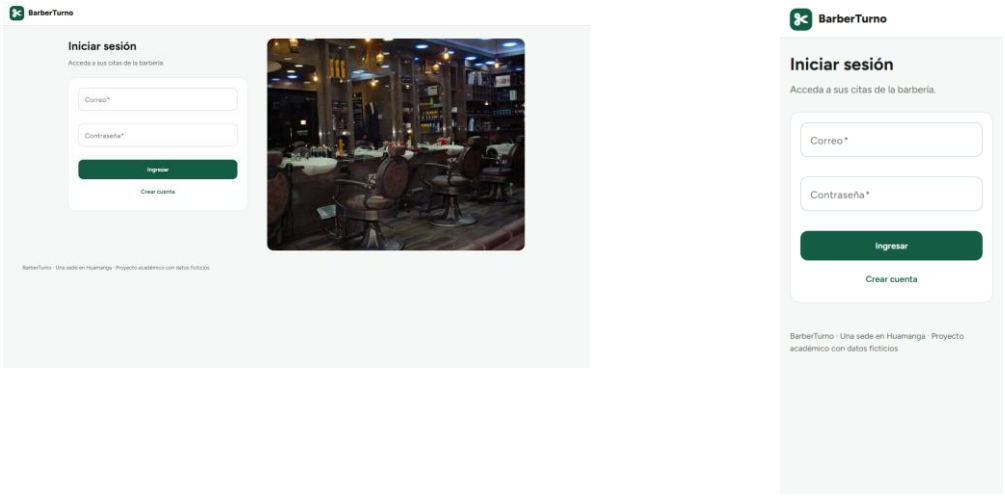



*Figura 34. Producción en Azure: página de ingreso en 1440 y 360 px. Capturas propias por GET público, sin iniciar sesión (21/10/2026).*

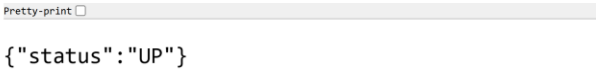



*Figura 35. GET /actuator/health en producción: 200 {"status":"UP"} (21/10/2026).*

**Comprobaciones con peticiones públicas (19/10 y 21/10/2026):** salud 200 UP; HTTP → 301 a HTTPS; HSTS, CSP, `X-Frame-Options: DENY`, `nosniff` y `Referrer-Policy`; cookie `XSRF-TOKEN` con `Secure; SameSite=Strict`; Swagger no público. El responsable inició sesión como administrador y entró en Agenda.

> **Diferencia con la entrega académica.** Azure ejecuta el jar de `d2f3107` (19/10/2026); la versión entregada es `3bcd5d0`, que añade solo T-58 (exportación a Excel y sus dependencias POI, Commons Lang y Guava). Por decisión del responsable no se redespliega por ahora: en producción no aparece el botón «Descargar Excel» y la demostración de T-58 se hace en local. Un redespliegue futuro requiere su autorización y repetir las comprobaciones de salud y cabeceras del registro de despliegue. Siguen pendientes CP-11, la medición de disponibilidad (RNF-02) y las pruebas operativas en producción.

# 13. Estado de aceptación, limitaciones y pendientes

La cobertura funcional implementada no equivale a la aceptación completa. La tabla separa lo acreditado de lo que todavía requiere evidencia.

| Elemento | Estado | Detalle |
|---|---|---|
| RF-01…RF-21 | Implementados y probados | Pruebas de integración y E2E en entorno local; CI en GitHub |
| CP-01…CP-19 (salvo CP-11) | Acreditados (local) | `docs/pruebas/aceptacion.md` |
| CP-20 (exportación) | Acreditado (T-58, en `main`) | `docs/pruebas/t-58.md` |
| CP-11 · usabilidad con cinco participantes (RNF-06) | Pendiente: 0/5 sesiones | Material preparado sobre la versión final; no se sustituye por E2E ni por el ensayo piloto |
| RNF-02 · 99 % de disponibilidad mensual | En observación, no cumplido | Monitor desde el 19/10/2026; periodo propuesto: noviembre de 2026 completo |
| Prueba mínima en producción (PM-1…PM-8) | Pendiente | Solo PM-0 (HTTPS y cabeceras) hecho; las escrituras requieren autorización del responsable |
| Restauración en Azure (RNF-09) | Preparada, no ejecutada | Ensayo local hecho (T-36); en Azure, procedimiento y coste estimados |
| RNF-07 · navegadores comerciales | Parcial | Chromium y Firefox de Playwright; falta la matriz de versiones comerciales |
| RNF-03 · HTTPS en producción | Cumplido | Azure *HTTPS Only* y HSTS |
| Validación con el negocio | Pendiente | Caso académico: reglas y precios propuestos, sin representante de una barbería real |

## 13.1 Limitaciones conocidas

- La reserva por autoservicio queda confirmada de inmediato; el flujo con confirmación manual existe como parámetro, pero el negocio no lo ha validado.
- No hay notificaciones por correo, SMS ni WhatsApp: los avisos son internos (alcance del MVP).
- La regla de red de la base depende de las IP salientes de App Service; si Azure las cambia, el monitor lo detectaría y habría que actualizar el firewall.
- `HEAD /` responde sin cabeceras de seguridad (GET es correcto); no afecta a los navegadores.
- La CSP admite `style-src 'unsafe-inline'` por compatibilidad con Angular Material; un nonce sería una mejora futura.

# 14. Autoría, uso de asistentes de IA y dominio del código

El proyecto se desarrolló con asistentes de IA bajo la dirección del estudiante, y así se declara. La rúbrica valora que el código sea del estudiante **o que este lo domine**; por eso esta sección explica cómo se repartió el trabajo y cómo se prepara la defensa.

| Participante | Papel |
|---|---|
| Sohnny Walter Lima Infanzón (responsable) | Define el problema y el alcance, toma las decisiones (alternativa, freeze, despliegue, excepciones como T-58), aprueba las integraciones, despliega en Azure, ejecutará CP-11 y sustenta. |
| Claude Code (Anthropic) | Arquitectura, requisitos, plan de tareas, revisión técnica de cada entrega y redacción de documentación; implementaciones pequeñas de coordinación. 103 commits de `main` llevan su coautoría explícita. |
| Codex (OpenAI) | Implementación principal del código y de las pruebas, mediante encargos con criterios de aceptación; sus commits figuran con la identidad Git del responsable. |
| Gemini (Google) | Revisión independiente, en solo lectura, de las tareas críticas. |

GitHub muestra a «sohnny-lima» y «claude» como colaboradores porque Git registra los autores y la coautoría. Para demostrar dominio, el guion de sustentación incluye el recorrido de una petición, las preguntas de defensa con respuestas basadas en el código real y los archivos que conviene abrir durante la exposición.

# 15. Propuesta de sustentación

El aula no fija una estructura ni una duración para la exposición; la siguiente es una **propuesta** de 10 a 12 minutos con una demostración de unos 4 minutos. El detalle, las frases clave y las preguntas de defensa están en `Guion_Sustentacion.md` y en la presentación `APF3_BarberTurno_Sustentacion.pptx`.

| Bloque | Minutos | Contenido |
|---|---|---|
| 1. Contexto y continuidad | 1 | Problema, alcance y paso del prototipo del APF2 al sistema |
| 2. Arquitectura y recorrido de una petición | 2 | Angular → Spring Security → controlador → servicio → repositorio → PostgreSQL |
| 3. Patrones y seguridad | 2 | MVC, DAO, SOLID, TDD (lo que sí y lo que no) y controles de seguridad |
| 4. Bibliotecas Java | 1,5 | Logback, POI, Commons Lang y Guava en el código de la exportación |
| 5. Demostración | 4 | Cliente reserva; barbero atiende; administrador gestiona, consulta reportes y exporta a Excel |
| 6. Git, pruebas y despliegue | 1,5 | Historial y CI en GitHub; resultados; Azure |
| 7. Pendientes y cierre | 0,5 | CP-11, RNF-02 y prueba en producción; conclusiones |

# 16. Conclusiones

- El diseño del APF2 se implementó sin cambiar el alcance: los catorce requisitos comprometidos funcionan para los tres roles y se añadieron siete mejoras trazables.
- La arquitectura aplica MVC en el backend Spring, DAO con repositorios Spring Data y principios SOLID verificables; TDD se aplicó y documentó en tareas concretas, sin atribuirlo al proyecto entero.
- La integridad de las reservas no depende de la interfaz: bloqueos ordenados, revalidación en la transacción y restricciones de exclusión en PostgreSQL lo garantizan, y las pruebas de concurrencia lo comprueban.
- Las cuatro bibliotecas de la rúbrica tienen un uso real y explicado; las tres incorporadas en T-58 se limitan a una función útil y protegen la información exportada.
- El historial de Git, la CI y las evidencias por commit permiten reproducir los resultados; el despliegue en Azure funciona con HTTPS y monitorización.
- Quedan pendientes la usabilidad con personas, la disponibilidad mensual y la prueba funcional en producción; se informan como tales.

# Referencias

Lima Infanzón, S. W. (2026). *BarberTurno: Avance de Proyecto Final 2* [Informe académico]. Universidad Tecnológica del Perú.

Universidad Tecnológica del Perú. (2026). *Rúbrica SI53-APF3-2026A: Avance de Proyecto Final 3*. Curso Integrador I: Sistemas Software.

Spring. (2026). *Spring Boot Reference Documentation 4.1*. https://docs.spring.io/spring-boot/

Spring. (2026). *Spring Security Reference: Cross Site Request Forgery*. https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html

PostgreSQL Global Development Group. (2026). *Range Types: Constraints on Ranges*. https://www.postgresql.org/docs/18/rangetypes.html

The Apache Software Foundation. (2025). *Apache POI: the Java API for Microsoft Documents*. https://poi.apache.org/

The Apache Software Foundation. (2025). *Apache Commons Lang*. https://commons.apache.org/proper/commons-lang/

Google. (2026). *Guava: Google Core Libraries for Java*. https://github.com/google/guava

QOS.ch. (2026). *Logback Project*. https://logback.qos.ch/

OWASP Foundation. (s. f.). *CSV Injection*. https://owasp.org/www-community/attacks/CSV_Injection

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

Martin, R. C. (2017). *Clean Architecture*. Prentice Hall.

Beck, K. (2002). *Test-Driven Development: By Example*. Addison-Wesley.

Lima Infanzón, S. W. (2026). *Repositorio BarberTurno* [Código fuente]. GitHub. https://github.com/sohnny-lima/BarberTurno

# Anexo A. Resumen de la matriz de la rúbrica

Autoevaluación preliminar del estudiante con la evidencia disponible; no sustituye la evaluación del docente. El detalle está en `Matriz_Rubrica.md`.

| Criterio | Máx. | Evidencia principal | Brecha pendiente |
|---|---|---|---|
| 1. Diseño de la solución | 3 | MVC, DAO y SOLID con clases concretas; TDD en T-49 y T-58; seguridad en el servidor | Ninguna esencial; TDD declarado como parcial |
| 2. Recursos Java | 2 | Logback (registro sin datos personales) y POI, Commons Lang y Guava en T-58, con pruebas | Ninguna esencial; producción sin T-58 |
| 3. Control de versiones | 3 | 436 commits en `main`, merges por tarea, CI, capturas auténticas | Ninguna esencial |
| 4. Interfaces gráficas | 6 | P01–P08 implementadas para los tres roles; E2E 48/48; capturas con axe 0 | Ninguna esencial para la cobertura; CP-11 (usabilidad) pendiente |
| 5. Construcción del producto | 4 | Completa, coherente, buenas prácticas; uso de IA declarado | El dominio del código debe demostrarse en la sustentación |
| 6. Sustentación oral | 2 | Presentación, guion y preguntas de defensa | Depende del ensayo y la exposición |

# Anexo B. Inventario de evidencias y reproducción

| Evidencia | Ubicación |
|---|---|
| Requisitos, arquitectura, tareas y decisiones | `docs/requisitos.md`, `docs/arquitectura.md`, `docs/tareas.md` |
| Aceptación CP-01…CP-19 y RNF | `docs/pruebas/aceptacion.md` |
| Capturas de la interfaz (auditadas con axe) | En la entrega, `evidencias/interfaces/` (89 capturas sobre `e0ebc54`, 21/10/2026, con su índice); históricas en el repositorio: `docs/pruebas/t-55/capturas/` y `docs/pruebas/t-57/capturas/` |
| Exportación a Excel (T-58) | `docs/pruebas/t-58.md` en la rama `tarea/T-58-exportar-excel`; en la entrega, `evidencias/t-58/` con `reporte-ejemplo.xlsx` (datos ficticios) y su vista en Excel |
| Despliegue | `docs/p-03-alojamiento.md` §12; `evidencias/azure/` |
| GitHub | https://github.com/sohnny-lima/BarberTurno y `evidencias/github/` |
| Diagramas editables | `fuentes/diagramas/*.svg` |

**Reproducir localmente** (JDK 21, Node 24.21.0 con fnm y PostgreSQL 18 en el puerto 5433): preparar la base con `tools/db-local.sql`; backend `cd backend && ./mvnw verify`; frontend `cd frontend && npm ci && npm run lint && npm test -- --watch=false && npm run build`; demostración con `./mvnw spring-boot:run -Dspring-boot.run.profiles=dev,demo` y `BT_DEMO_PASSWORD`. Los detalles están en el `README.md` del repositorio y de la entrega.